



K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

“Linux Memory-Mapped File Management System”

An Operating Systems Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

Santosh kumar bal -2520030474

K.Charan tej -2520030530

G.Bhargav -2520030451

Under the Esteemed Guidance of

Hema

Department of Computer Science and Engineering

Subject: Operating Systems

Academic Year: 2026



DECLARATION

The Project Report entitled “**Linux Memory-Mapped File Management System**” is a record of bonafide work of **Santosh kumar bal -2520030474 ,K.Charan tej -2520030530 and G.Bhargav -2520030451**, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to K L (Deemed to be) University. The work was carried out for the subject Operating Systems. The results embodied in this report have not been copied from any other department, university or institute.

Place: [PLACE]

Date: [DATE]

Signature of the Student



K L (Deemed to be) University

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled “**Linux Memory-Mapped File Management System**” is a bonafide work done and submitted by **Santosh kumar bal -2520030474 ,K.Charan tej -2520030530 ,G.Bhargav -2520030451** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be University), for the subject **Operating Systems**, during the academic year 2026

Signature of the Guide

**Signature of the Course
Coordinator**

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, support and encouragement of many people. I take this opportunity to express my sincere thanks to all those who helped me in one way or another during the work on this project.

I express my deep gratitude to my project guide, **Hema**, Department of Computer Science and Engineering, and to my Course Coordinator, [**COURSE COORDINATOR**], for their valuable guidance, constant encouragement and suggestions, which helped me to understand the concepts of Operating Systems and to complete the project in a systematic manner.

I am thankful to **HOD sir** Head of the Department of Computer Science and Engineering, for providing the facilities and the academic environment needed for the completion of this project.

I also thank the teaching and non-teaching staff of the department, and K L (Deemed to be) University, for their support. Finally, I sincerely thank my parents, friends and classmates for their help, patience and cooperation throughout this work.

Santosh kumar bal -2520030474

K.Charan tej -2520030530

G.Bhargav -2520030451

ABSTRACT

Operating systems provide file access, memory management and protection through system calls, yet students often study these topics separately and find it difficult to see how they are connected in a running program. This project, the Linux Memory-Mapped File Management System, addresses this gap with a user-space program written in C for Linux, developed and executed on Ubuntu, including Ubuntu running under Windows Subsystem for Linux (WSL).

Traditional file access uses `read()` and `write()`, which copy data between the kernel's page cache and a buffer in the process and require a system call for every transfer. Memory-mapped file I/O offers an alternative. A file is associated with a range of the process's virtual address space by `mmap()`, and the program accesses it with ordinary memory operations. The kernel loads pages on demand through page faults instead of copying the whole file into memory, and the mapped pages are backed by the file.

The program implements a complete workflow. It opens a file with `open()`, obtains its size and type with `fstat()`, maps it as a shared, readable and writable region with `mmap()`, displays the file and mapping information, and accesses and modifies the content through the mapped address. It then requests synchronization of the changes with the file using `msync()`, removes the mapping with `munmap()` and releases the file descriptor with `close()`. Every call is checked for failure, errors are reported using `errno`, and resources are released in the reverse order of their acquisition.

The report explains the related concepts: the process address space, virtual and physical memory, pages, demand paging, the page cache, shared and private mappings, protection flags, synchronization and persistent storage. It also describes the architecture, modules, algorithm, execution procedure and test cases. Performance is discussed conceptually, because no quantitative benchmarks were collected, and the report does not claim that memory mapping is always faster.

The project does not modify the kernel or implement a virtual-memory subsystem. Its significance is educational: it turns abstract operating-system concepts into observable behaviour that can be demonstrated, inspected with tools such as `/proc/<pid>/maps`, and explained in a viva.

INDEX

S.No.	Chapter	Topics / Subtopics	Page No.
	Declaration		i
	Certificate		ii
	Acknowledgement		iii
	Abstract		iv
1	Introduction		1
		1.1 Background of the Project	1
		1.2 Problem Statement	2
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	3
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	4
2	OS Concepts and Background Study		5
		2.1 Operating System Concepts Related to the Project	5
		2.2 Traditional File I/O vs Memory-Mapped File I/O	9
		2.3 Existing Methods / Approaches	11

		2.4 Proposed Approach	11
		2.5 Advantages of the Proposed Approach	12
		2.6 Limitations of the Proposed Approach	12
3	System Requirements		14
		3.1 Hardware Requirements	14
		3.2 Software Requirements	14
		3.3 Operating System Environment — Ubuntu / WSL	15
		3.4 Development Tools and Compiler	16
		3.5 Libraries and System APIs	16
4	Technology and Tools Used		17
		4.1 C Programming	17
		4.2 GCC Compiler	17
		4.3 Linux System Calls	17
		4.4 mmap() and Memory Mapping	18
		4.5 munmap() and Resource Release	19
		4.6 msync() and Memory Synchronization	19

		4.7 File Operations — open(), close(), fstat()	19
		4.8 Ubuntu / WSL	20
		4.9 Visual Studio Code	20
		4.10 Linux Commands and Utilities	20
5	System Architecture and Design		22
		5.1 Overall System Architecture	22
		5.2 System Workflow	23
		5.3 Data Flow	24
		5.4 Memory Mapping Workflow	25
		5.5 Module Description	28
		5.6 Input Design	30
		5.7 Output Design	30
		5.8 Error Handling Design	31
		5.9 Resource Management Design	32
6	Implementation		33

		6.1 Module-wise Implementation	33
		6.2 File Opening Module	33
		6.3 File Information Module	34
		6.4 Memory Mapping Module	35
		6.5 Mapped Memory Access Module	36
		6.6 Memory Synchronization Module	36
		6.7 Memory Unmapping Module	37
		6.8 File Closing Module	37
		6.9 Error Handling	37
		6.10 Algorithm	38
		6.11 Pseudocode	38
		6.12 Core C Implementation	39
		6.13 Linux System Calls / APIs Used	42
7	Execution and Demonstration		44
		7.1 Compilation in Ubuntu / WSL	44
		7.2 Execution Commands	44

		7.3 Sample Input	45
		7.4 Sample Output	45
		7.5 Demonstration of mmap()	46
		7.6 Demonstration of File Modification	46
		7.7 Demonstration of msync()	47
		7.8 Demonstration of munmap()	47
		7.9 Error Cases	48
		7.10 Screenshots to be Included	50
		7.11 Demonstration of OS Concepts	50
8	Results, Conclusion and Future Scope		52
		8.1 Summary of the Project	52
		8.2 Objectives Achieved	52
		8.3 Results Achieved	53
		8.4 Operating System Concepts Demonstrated	53
		8.5 Skills and Learning Outcomes	54
		8.6 Limitations	54

		8.7 Future Enhancements	55
		8.8 Conclusion	55
9	References		57
		9.1 Books	57
		9.2 Research Papers	57
		9.3 Linux / GNU Documentation	57
		9.4 Online Resources	58
		Information still to be filled in	60
		Screenshots to capture	60
		Diagrams to create or check	60
		Code snippets to include	60
		Test results to verify	61
		Final formatting steps	61

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

An Operating System (OS) is the system software that manages the hardware of a computer and provides services to application programs. Among the most important services it offers are process management, memory management, file management and device management. Application programs do not access the disk or physical memory directly. They request these services from the kernel through a controlled interface known as system calls.

Linux is a widely used open-source, Unix-like operating system. Its kernel provides a rich set of system calls through which programs written in C can create processes, open files, allocate memory and communicate with other processes. Because Linux exposes these mechanisms in a clear and well-documented way, it is a very suitable platform for studying how an operating system behaves internally.

Every running program in Linux is a process, and every process is given its own private virtual address space. A virtual address is the address that a program uses in its instructions and pointers. The hardware memory management unit (MMU), under the control of the kernel, translates virtual addresses into physical addresses in main memory. This arrangement allows each process to behave as if it owns a large continuous block of memory, while the kernel keeps processes isolated from one another.

The usual way of working with a file in C on Linux is to call `open()` and then to transfer data between the file and a buffer in the program using `read()` and `write()`. Linux also offers a second approach, called memory-mapped file I/O. With the `mmap()` system call, the kernel associates a range of the process's virtual address space with a file. After the mapping has been created, the program reads and writes the file contents by using ordinary memory operations such as pointer dereferencing, array indexing and `memcpy()`.

This project, titled **Linux Memory-Mapped File Management System**, is a Linux user-space program written in C that demonstrates this mechanism step by step. It opens a file, inspects it with `fstat()`, maps it with `mmap()`, accesses and modifies the contents through a memory address, synchronizes the changes with `msync()`, removes the mapping with `munmap()` and finally releases the file descriptor with `close()`. The project is intended as a practical illustration of how several operating-system ideas (virtual memory, paging, the page cache, file descriptors and system calls) come together in one real facility.

Scope clarification. This project is a user-space application that uses Linux system calls. It does not modify the Linux kernel, it does not implement a virtual-memory subsystem, and it is not itself an operating system. The virtual-memory behaviour described in this report is carried out by the Linux kernel; the project demonstrates and observes that behaviour through the standard programming interface.

1.2 Problem Statement

Students of Operating Systems usually learn virtual memory, paging and file systems as separate theoretical topics. As a result, it is often difficult to see how a file on disk, a region of a process's virtual address space and the pages managed by the kernel are connected in practice. Conventional file programs that rely only on `read()` and `write()` hide this connection, because the data is always copied between kernel-managed memory and a user-space buffer.

The problem addressed by this project is therefore to design and implement a Linux program that makes this relationship visible. The program must map a file into the virtual address space of its process, show the details of the mapping, allow the file contents to be read and modified through memory, make sure that the

modifications reach the underlying file, and release every resource correctly. The program must also handle failure situations such as a missing file, an empty file or insufficient permissions in a controlled way, instead of terminating abnormally.

1.3 Scope of the Project

The scope of the project is limited to the demonstration of memory-mapped file management in a user-space Linux environment. Within this scope the project covers the following activities:

- Accepting the name of a file that is to be mapped.
- Opening the file and obtaining its metadata, in particular its size, using `fstat()`.
- Mapping the file into the process's virtual address space with `mmap()` using a shared, readable and writable mapping.
- Displaying relevant file and mapping information.
- Accessing and modifying the mapped content through a memory address.
- Requesting synchronization of the modified pages with the file using `msync()`.
- Removing the mapping with `munmap()` and closing the file descriptor with `close()`.
- Reporting success and failure conditions through meaningful messages.

The project does not attempt to implement a text editor, a database, a file system, a kernel module, a memory allocator or a complete virtual-memory manager. It also does not claim any measured performance improvement. The report discusses performance only in a conceptual manner (see Section 2.2.3).

Note on the user interface. This report documents the program as a command-line application that takes the file name from a command-line argument or from a prompt and then proceeds through the workflow in a fixed sequence. If the submitted source code contains a menu-driven interface, Sections 5.6, 6.5 and 7.3 should be updated accordingly. A menu-driven interface is not claimed in this report.

1.4 Objectives of the Project

The main objective of the project is to design and implement a Linux-based memory-mapped file management system that demonstrates how files can be accessed and modified through memory mapping. The specific objectives are listed below.

1. To study the relationship between a process's virtual address space, file-backed memory and the underlying file.
2. To implement the complete life cycle of a file mapping: open, inspect, map, access, modify, synchronize, unmap and close.
3. To demonstrate the use of the system calls `open()`, `fstat()`, `mmap()`, `msync()`, `munmap()` and `close()` with correct parameters and return-value checking.
4. To show that a mapped file can be read and modified as ordinary memory without explicit `read()` and `write()` calls for the data transfer.
5. To explain the difference between shared and private mappings and the role of protection flags.
6. To demonstrate safe resource management, so that no file descriptor or mapping is left open after the program finishes.
7. To provide a well-documented example that can be used to explain virtual memory, paging and the page cache during a viva or a laboratory demonstration.

1.5 Importance of the Application

Memory-mapped file I/O is not only an academic topic. It is used in real software for loading executables and shared libraries, for implementing shared memory between processes, for working with large data files and for building database storage engines and caching layers. The Linux kernel itself loads program code into a process by mapping the executable file. Understanding `mmap()` therefore gives a student a view into one of the central mechanisms of the operating system.

The project is important from an educational point of view for the following reasons. First, it connects several theoretical concepts (virtual memory, demand paging, page faults, the page cache and file descriptors) to one concrete program that can be compiled, run and observed. Second, it shows the correct sequence in which system calls must be used and how each call can fail. Third, it introduces good system-programming practice: checking every return value, reporting errors with `errno`, and releasing resources on every exit path. Fourth, the program can be examined with standard Linux tools such as `stat`, `cat`, `ls -l` and `/proc/<pid>/maps`, which makes the OS concepts visible during a demonstration.

1.6 Target Users / Audience

The primary audience of this project consists of B.Tech Computer Science and Engineering students who are studying Operating Systems and wish to understand memory management through a hands-on example. The project is also useful for the following groups:

- Course instructors and project evaluators, who can use the program and the report to assess the student's understanding of virtual memory and file-backed mappings.
- Beginners in Linux system programming who want a small, complete example of the `mmap()` family of system calls.
- Students who are preparing for viva examinations or technical interviews on operating systems.

No special background beyond basic C programming, familiarity with a Linux terminal and an introductory knowledge of operating-system concepts is required to understand or run the project.

CHAPTER 2 — OS CONCEPTS AND BACKGROUND STUDY

This chapter explains the operating-system concepts on which the project is based. Each concept is described in general and then related to the specific part of the project where it appears. The chapter also compares traditional file I/O with memory-mapped file I/O, reviews existing approaches, and states the advantages and limitations of the approach used in this project.

2.1 Operating System Concepts Related to the Project

2.1.1 Operating System and Linux

An operating system acts as an intermediary between application programs and the computer hardware. It allocates the processor, memory and storage among programs and enforces protection so that one faulty program cannot damage another. Linux is an operating system kernel, together with the surrounding GNU and other utilities, that follows the POSIX model closely. In this project the Linux kernel is the component that

performs the real work of creating mappings, handling page faults and writing modified pages back to storage. The C program only requests these services.

2.1.2 User Space, Kernel Space and System Calls

The processor executes code in at least two privilege levels. Kernel code runs in a privileged mode and can access all hardware and memory. Application code runs in an unprivileged mode, called user mode, and can access only its own memory. The two environments are commonly referred to as kernel space and user space. A program in user space cannot directly manipulate page tables or disk controllers. It must ask the kernel to do so through a system call, which is a controlled entry into the kernel.

The functions `open()`, `fstat()`, `mmap()`, `msync()`, `munmap()` and `close()` are thin wrappers provided by the C library (glibc) around the corresponding Linux system calls. When one of them is invoked, the processor switches into kernel mode, the kernel performs the requested operation, and control returns to the program with a return value. On failure the wrapper returns -1 (or `MAP_FAILED` in the case of `mmap()`) and stores a code in the global variable `errno`. This project is written entirely in user space and relies on these calls for every kernel service.

2.1.3 Process and Process Address Space

A process is a program in execution. In addition to the program code, a process includes its data, its stack, the processor state and a set of kernel-maintained resources such as open file descriptors. Each process has its own virtual address space, which is divided into regions with different purposes: the text segment holding the machine code, the initialized and uninitialized data segments, the heap used for dynamic allocation, the stack, and areas used for shared libraries and memory mappings. The kernel records these regions as a set of virtual memory areas, and each area has a start address, an end address, access permissions and, for a file-backed mapping, a reference to the file and the offset within it.

When `mmap()` succeeds, a new virtual memory area is added to the address space of the process. Its permissions and its link to the file can be observed at run time in the file `/proc/<pid>/maps`, where a shared read-write file mapping appears with the permission string `rw-s`.

2.1.4 Virtual Memory, Virtual Address and Physical Memory

Virtual memory is the technique by which the operating system gives each process the illusion of a large, private and continuous memory, regardless of the amount of physical memory installed. Addresses seen by the program are virtual addresses. Physical memory is the actual RAM of the machine. The mapping between virtual pages and physical frames is stored in page tables maintained by the kernel and used by the MMU. A virtual page need not be present in physical memory at all times, and several processes may map the same physical frame at different virtual addresses.

The pointer returned by `mmap()` in this project is a virtual address. It has no fixed relationship with the physical location of the file data, and it may differ from one run of the program to the next because of address-space layout randomization.

2.1.5 Memory Pages, Page Faults and Demand Paging

Virtual memory is divided into fixed-size blocks called pages, and physical memory is divided into frames of the same size. On most Linux systems for the x86-64 architecture the page size is 4096 bytes, and it can be queried with `sysconf(_SC_PAGESIZE)` or `getconf PAGESIZE`. The kernel manages memory in units of whole pages. A mapping always occupies a whole number of pages even if the file is smaller than that.

Under demand paging a page is brought into physical memory only when it is first needed. When a program accesses a virtual address whose page is not yet present in its page table, the MMU raises a page fault. The kernel then identifies the memory area that contains the address, finds or loads the required data, updates the page table and resumes the program. If the data is already in memory (for example in the page cache because another process or an earlier access loaded it), the fault is called a minor fault. If the data must be read from storage, it is a major fault. For a file-backed mapping, the data is obtained from the file.

Therefore, calling `mmap()` on a large file does not copy the whole file into RAM. The call establishes the mapping, and the contents are brought into memory page by page as the program touches them. The kernel may also read additional pages ahead of the access (read-ahead) and may later reclaim clean pages when memory is needed.

2.1.6 File Descriptors and the File Operations

A file descriptor is a small non-negative integer that identifies an open file within a process. It is an index into a per-process table of open files and is returned by `open()`. The descriptors 0, 1 and 2 are normally used for standard input, standard output and standard error, so the first descriptor returned to a program is usually 3. In the sample run of this project the descriptor value 3 is reported.

It is important to distinguish a file descriptor from a memory address. The file descriptor is a handle used for system calls such as `fstat()` and `close()` and has no meaning as a pointer. The address returned by `mmap()` is a virtual address that can be dereferenced directly. After a mapping has been established, the mapping remains valid even if the descriptor is closed, because the kernel keeps its own reference to the file. This project nevertheless closes the descriptor only after `munmap()` to keep the sequence of operations clear and symmetrical.

The system call `open()` opens an existing file and returns a descriptor. `fstat()` returns information about the file referred to by the descriptor, such as its size, type, permission bits and inode number, in a `struct stat`. `close()` releases the descriptor so that it can be reused and so that the kernel can free the associated resources when no references remain.

2.1.7 File-Backed Memory and the Page Cache

File-backed memory is a region of virtual memory whose contents correspond to a file instead of to anonymous (zero-filled) memory. Linux keeps recently used file data in memory in a structure called the page cache. Both ordinary `read()` and `write()` calls and file-backed mappings use the page cache. When a page of a mapped file is accessed, the kernel makes the cached page available to the process by placing it in the process's page table. A modification to a shared mapping changes the cached page directly, and the kernel marks the page as dirty so that it will eventually be written to storage.

2.1.8 Memory Mapping with `mmap()`

The call `mmap()` creates a new mapping in the virtual address space of the calling process. Its prototype is shown below.

Prototype of `mmap()`

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot,

           int flags, int fd, off_t offset);
```

The argument `addr` is a hint for the preferred address; passing `NULL` lets the kernel choose a suitable address. The argument `length` is the number of bytes to map and must be greater than zero. The argument `prot` specifies the protection of the mapping and is a combination of `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` or `PROT_NONE`. The argument `flags` selects the type of the mapping. The arguments `fd` and `offset` identify the file and the starting position in the file; `offset` must be a multiple of the page size. On success the call returns the starting virtual address of the mapping, and on failure it returns `MAP_FAILED`, which is the value `(void *) -1`.

`mmap()` is different from `malloc()`. The function `malloc()` is a library routine that manages dynamic memory in the heap and requests memory from the kernel only indirectly, and the memory it returns has no relationship with a file. `mmap()` is a system call that creates a mapping of a chosen kind, and with a file descriptor it creates memory that is backed by that file.

2.1.9 Shared and Private Mappings and Protection Flags

The `flags` argument must specify either `MAP_SHARED` or `MAP_PRIVATE`. In a shared mapping, updates to the mapped memory are visible to other processes that map the same file region and are carried through to the underlying file. In a private mapping, the first write to a page causes the kernel to create a private copy of that page (copy-on-write); the changes are visible only to the writing process and are not carried to the file. This project uses `MAP_SHARED` because its purpose is to modify the file through memory.

The protection flags control how the mapped memory may be accessed. This project requests `PROT_READ | PROT_WRITE`. A write-enabled shared mapping requires that the file descriptor was opened with the `O_RDWR` mode; otherwise `mmap()` fails with the error `EACCES`. If the program attempts an access that the protection does not allow, the processor raises a fault and the process receives the signal `SIGSEGV`. An access to a page of the mapping that lies completely beyond the end of the file results in the signal `SIGBUS`, and this is the reason the project limits every write to the mapped length.

2.1.10 Memory Synchronization, `msync()` and Persistent Storage

Persistent storage is storage such as a hard disk or SSD whose contents survive after the program ends and after the machine is powered off. Main memory, in contrast, is volatile. Modifications made through a shared mapping first change pages in memory. The kernel writes dirty pages to the file in the background at times chosen by its own policy. A program that needs to request that its changes be written at a specific point can call `msync()`.

Prototype of `msync()`

```
int msync(void *addr, size_t length, int flags);
```

The argument `addr` must be page-aligned and `length` gives the size of the region. With the flag `MS_SYNC` the call requests that the modified pages be written to the file and does not return until the operation is complete. With `MS_ASYNC` the write-back is scheduled and the call returns without waiting; its effect on modern Linux kernels is limited, and the manual page should be consulted for details. The flag `MS_INVALIDATE` asks for other mappings of the same file to be invalidated so that they obtain fresh data. The function returns 0 on success and -1 on error. Because the exact behaviour depends on the mapping type and the flags, this report describes `msync()` as a request to synchronize mapped modifications with the underlying file, and this project uses `MS_SYNC` so that the call returns only after the request has been carried out.

2.1.11 `munmap()`, Error Handling and Resource Management

The call `munmap(addr, length)` removes the mapping of the given range from the address space of the process. After the call, any access to that range is an invalid memory access. It does not delete the file or the data already written to it. The call returns 0 on success and -1 on error. Terminating the process also removes all of its mappings and closes all of its descriptors automatically, but good practice requires that a program release the resources explicitly when they are no longer needed. This is especially important for long-running programs, where unreleased descriptors and mappings would accumulate.

Error handling means that every system call is checked for failure and that the program reacts in a controlled way. In this project, each call is checked. On failure, the program prints a message that includes the text obtained from `strerror(errno)`, releases whatever resources have been acquired so far, and exits with a failure status. This pattern of acquiring resources in order and releasing them in the reverse order is the central resource-management principle of the program.

2.2 Traditional File I/O vs Memory-Mapped File I/O

2.2.1 Traditional File I/O

In traditional file I/O, the program allocates a buffer in user space and calls `read(fd, buffer, count)` to ask the kernel to copy data from the file into the buffer. To modify the file, the program changes the buffer and calls `write(fd, buffer, count)` to copy the data back to the kernel, which then writes it to the file. Random access requires `lseek()` or the calls `pread()` and `pwrite()`. In each transfer the data is copied between the kernel's page cache and the user buffer, and each call involves a transition from user mode to kernel mode.

2.2.2 Memory-Mapped File I/O

In memory-mapped file I/O, the program asks the kernel once to map the file. The data is then accessed through normal memory instructions. The page cache pages are made directly visible in the process's address space, so the explicit copy into a user buffer is not needed. A kernel transition occurs when the mapping is created, when a page fault must be handled, when `msync()` is called and when the mapping is removed, but not for each individual access to data that is already mapped and present. Table 2.1 compares the two approaches.

Table 2.1: Traditional File I/O vs Memory-Mapped I/O

Aspect	Traditional read()/write()	Memory-mapped I/O (mmap)
Access method	Explicit system calls to transfer data	Pointer or array access in the address space
Data location	Copied between page cache and a user-space buffer	Page-cache pages mapped into the process address space
System calls per access	One call per transfer	No call for access to pages already mapped and resident
Random access	Needs lseek(), pread() or pwrite()	Index or pointer arithmetic
Loading of data	Data read when requested by read()	Pages loaded on demand through page faults
Modification	Edit buffer, then write()	Modify memory directly; msync() requests write-back
File size change	write() can extend a file	The mapping length is fixed; the file must be resized separately (for example with ftruncate())
Error indication	Return value of read() and write()	Return value of mmap(), but invalid access gives SIGSEGV or SIGBUS
Suitable for	Sequential streams, sockets, small transfers	Large files with random access, shared data, loading of executables
Portability	Standard on all platforms	POSIX systems; behaviour differs on other systems

2.2.3 Conceptual Performance Considerations

No quantitative benchmark results were collected in the current implementation; therefore, performance claims are discussed conceptually rather than numerically.

Conceptually, memory mapping can reduce explicit copying between user-space buffers and the kernel-managed file I/O path in suitable workloads, because the process accesses the cached file pages directly through its virtual address space. It can also reduce the number of system calls when a program accesses many small pieces of a file at random positions, since each access to an already-resident page is an ordinary memory operation. The first access to each page, however, may cause a page fault, and a major page fault requires a read from storage, which is much slower than a memory access.

The actual performance therefore depends on several factors: the size of the file, the access pattern (sequential or random), whether the data is already in the page cache, the characteristics of the storage device, the number of pages touched and the overhead of handling page faults. For small files or for strictly sequential reading, `read()` and `write()` may be equally fast or even faster. It is therefore incorrect to state that `mmap()` automatically makes every program faster, and this project does not make such a claim.

2.3 Existing Methods / Approaches

Several approaches exist for accessing the contents of a file from a C program on Linux.

- **System-call I/O with `read()` and `write()`.** This is the lowest-level standard approach and uses file descriptors directly. It gives full control but requires explicit buffer management.
- **Standard C library I/O (`fopen()`, `fread()`, `fwrite()`, `fgets()`).** The C library adds a user-space buffer on top of the system calls to reduce the number of calls. It is convenient and portable.
- **Positioned I/O with `pread()` and `pwrite()`.** These calls read or write at a given offset without changing the file position and are useful for random access and multi-threaded programs.
- **Direct I/O (`O_DIRECT`).** This approach bypasses the page cache and is used by specialised programs such as databases that manage their own caching.
- **Memory-mapped I/O with `mmap()`.** The file is mapped into the address space and accessed as memory, as in this project.
- **Higher-level libraries and tools.** Text editors and database engines use combinations of the above approaches internally.

In many existing examples found in textbooks and tutorials, `mmap()` is shown only as a short code fragment that maps and prints a file. Such examples usually do not show file inspection, error handling for each stage, synchronization, or an orderly release of resources.

2.4 Proposed Approach

The proposed approach is a structured user-space C program that follows the complete life cycle of a file mapping in a fixed order and checks every step. The file is opened in read-write mode, its size is obtained with `fstat()`, and a shared read-write mapping covering the whole file is created. The program displays the file information and the mapping information, including the virtual address, the length, the page size and the number of pages spanned. It then shows the mapped content, accepts a modification and applies it directly to the mapped memory. The program calls `msync()` to request that the change be written to the file, removes the mapping with `munmap()` and closes the descriptor with `close()`.

The program is organised into small modules, each responsible for one system-call stage (Chapter 5). Each module checks the result of its call and passes control to the next stage only if the call has succeeded. In this

way the structure of the source code reflects the structure of the underlying kernel mechanism, which makes the program suitable as a teaching example.

2.5 Advantages of the Proposed Approach

- The relationship between a file, a virtual address range and the kernel's page management becomes visible and can be observed with standard Linux tools.
- The file is accessed through ordinary memory operations, which removes the need for an explicit buffer and for `read()` and `write()` calls for the data transfer.
- The kernel loads pages on demand, so the program does not need to read the whole file in advance.
- The approach demonstrates in a single program several system calls, their parameters and their error codes.
- The strict ordering of the acquisition and release of resources demonstrates good system-programming discipline.
- The program is small and portable across Linux distributions, including Ubuntu on WSL, and uses only standard C and POSIX interfaces.

2.6 Limitations of the Proposed Approach

- The mapping length is fixed at the time of mapping. The program cannot grow the file by writing beyond its end, and an empty file cannot be mapped because a zero-length mapping is invalid.
- Modifications are limited to the size of the existing file; the program must prevent writes beyond the mapped length.
- Access errors on a mapping are reported as signals (`SIGSEGV`, `SIGBUS`) and not as return values, so incorrect programs fail more abruptly than with `read()` and `write()`.
- If another process truncates the file while it is mapped, accessing the removed part can raise `SIGBUS`.
- Memory mapping does not guarantee better performance; the result depends on the workload.
- The program demonstrates a single mapping of one file at a time and does not provide concurrent access control between several processes or threads.
- The project is an educational user-space application and does not implement kernel-level memory management.

Table 2.2: Advantages and Limitations of the Proposed Approach

Aspect	Advantage	Limitation
Data access	Direct pointer access to file data	Invalid access produces signals, not error codes
Memory use	Pages loaded on demand; no need to load the whole file	Large mappings still consume virtual address space
Modification	Shared mapping changes the file contents	Cannot extend the file through the mapping
Synchronization	<code>msync()</code> gives control over write-back timing	Behaviour depends on flags and mapping type
Learning value	Shows virtual memory, paging and the page cache	Does not implement them; they are performed by the kernel
Performance	Can avoid explicit copying in suitable workloads	Not guaranteed; no benchmarks collected in this project

CHAPTER 3 — SYSTEM REQUIREMENTS

This chapter lists the hardware and software required to compile, run and demonstrate the project. The project is small and does not need specialised equipment, but the environment must be a Linux system, since the program uses Linux system calls that are not available on plain Windows.

3.1 Hardware Requirements

The program itself requires very little processing power or memory. The figures in Table 3.1 are the practical requirements for a laptop or desktop computer on which Ubuntu (natively or under WSL) and Visual Studio Code are used together.

Table 3.1: Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	64-bit x86-64 processor, dual core	Quad-core 64-bit processor
Main memory (RAM)	4 GB	8 GB or more (needed when WSL and VS Code run together)
Storage	5 GB free disk space	10 GB or more free disk space (SSD preferred)
Display	1366 x 768 resolution	1920 x 1080 resolution
Input devices	Keyboard	Keyboard and mouse
Machine used by the student	[TO BE FILLED: processor, RAM and storage of the machine actually used]	

The requirements are not limited by the size of the file that is mapped. Since a mapping uses virtual address space and loads pages on demand, a file can be larger than the free physical memory, although the performance of access to such a file depends on the speed of the storage device.

3.2 Software Requirements

Table 3.2: Software Requirements

Software	Version / Details	Purpose
Operating system	Ubuntu Linux (native), or Ubuntu running under Windows Subsystem for Linux (WSL)	Provides the Linux kernel interface and system calls
Compiler	GCC (GNU Compiler Collection), installed with the build-essential package	Compiles the C source file into an executable
C library	GNU C Library (glibc)	Provides wrappers for the system calls and standard functions
Code editor	Visual Studio Code (with the WSL extension when WSL is used)	Writing and editing the source code
Shell	Bash	Compiling and running the program and using Linux utilities
Utilities	cat, ls, stat, getconf, echo, chmod, ps, xxd or hexdump (optional)	Preparing test files and verifying results
Optional tools	strace, gdb	Tracing system calls and debugging (optional)

The exact versions installed on the student's machine should be obtained by running `lsb_release -a` and `gcc --version` and entered in the report where indicated in the Document Preparation Checklist.

3.3 Operating System Environment — Ubuntu / WSL

The project is developed and executed in an Ubuntu Linux environment. Ubuntu is a widely used Linux distribution based on Debian. It provides the GNU toolchain, the standard system header files and the `/proc` file system that the project uses for observation.

Many students use Windows as the host operating system. Windows Subsystem for Linux (WSL) allows a Linux distribution such as Ubuntu to run on Windows without a separate virtual-machine installation. In WSL version 2, the Linux distribution runs on a real Linux kernel provided by Microsoft in a lightweight virtual machine, and Linux system calls such as `mmap()` are handled by that kernel. A distribution is started by opening an Ubuntu terminal, and Visual Studio Code can be connected to it through the WSL extension.

The following practical points apply when the project is developed under WSL.

- The source file and the files to be mapped should be kept in the Linux file system (for example under the home directory `~`) and not under `/mnt/c`, which is a view of the Windows drives. Files on the Windows drives are accessed through a translation layer, and behaviour such as permissions and performance can differ from that of a native Linux file system.
- The command `uname -r` shows the kernel version in use, and `getconf PAGESIZE` shows the page size that the mapping will use.
- File permissions are those of the Linux file system, so permission-related error cases can be reproduced with `chmod`.

The same source code compiles and runs without change on a native Ubuntu installation.

3.4 Development Tools and Compiler

The source code is written in C and compiled using GCC. GCC is installed on Ubuntu with the following commands.

Installing and verifying the compiler

```
sudo apt update  
  
sudo apt install build-essential  
  
gcc --version
```

The package `build-essential` installs GCC, the `make` utility and the standard development headers. The compiler options normally used are `-Wall` and `-Wextra`, which enable warnings that help to find mistakes such as unused variables and incorrect types, and `-g`, which includes debugging information. Visual Studio Code is used as the editor, and the program is compiled and run from its integrated terminal.

3.5 Libraries and System APIs

The program uses only the standard C library and the POSIX/Linux system-call interface. No third-party library is required. Table 3.3 lists the header files used and the facilities they provide.

Table 3.3: Header Files and Facilities Used

Header file	Facilities used in the project
<stdio.h>	printf(), fprintf(), fgets(), fwrite(), fflush(), perror-style messages
<stdlib.h>	EXIT_SUCCESS, EXIT_FAILURE
<string.h>	strncpy(), strlen-type helpers, strcspn(), memcpy(), strerror()
<errno.h>	errno, the variable that holds the error code of a failed call
<limits.h>	PATH_MAX, the maximum length of a path name
<fcntl.h>	open() and the flag O_RDWR
<unistd.h>	close(), sysconf(), getpid()
<sys/types.h>	Types such as off_t and pid_t
<sys/stat.h>	fstat(), struct stat, S_ISREG()
<sys/mman.h>	mmap(), munmap(), msync(), PROT_* and MAP_* constants, MS_SYNC

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

This chapter describes the programming language, the compiler, the system calls and the other tools that are used in the project, with emphasis on their role in the implementation.

4.1 C Programming

C is a general-purpose, procedural programming language that was developed together with the Unix operating system. The Linux kernel and most Linux system libraries are written in C. The language provides pointers, which store memory addresses, and gives the programmer direct control over memory. These properties make C the natural language for system programming. In this project, the pointer returned by `mmap()` is stored in a

`char * variable`, and the file contents are read and written through this pointer with array indexing, pointer arithmetic and `memcpy()`.

The project uses only standard C features: functions, structures (`struct stat`), pointers, strings and conditional statements. The return value of every system call is stored in a variable and checked immediately, which is the usual style in C system programming.

4.2 GCC Compiler

GCC, the GNU Compiler Collection, is the standard compiler on Linux. It translates the C source file into object code and links it with the C library to produce an executable file. The command used in this project has the following general form.

Compilation command (placeholders)

```
gcc -Wall -Wextra -g [SOURCE_FILE].c -o [EXECUTABLE_NAME]
```

The option `-o` gives the name of the output file. When the compilation succeeds without errors, the executable is run from the current directory using `./[EXECUTABLE_NAME]`. GCC reports errors and warnings together with the file name and line number, which makes it easy to correct the source code.

4.3 Linux System Calls

System calls are the interface between user programs and the kernel. The project uses six calls, summarised in Table 4.1. Their purposes are explained further in Table 4.2.

Table 4.1: System Calls Used in the Project

System call	Header	Prototype	Return value on success	Failure indication
open()	<fcntl.h>	int open(const char *path, int flags)	File descriptor (non-negative integer)	-1, errno set
fstat()	<sys/stat.h>	int fstat(int fd, struct stat *buf)	0	-1, errno set
mmap()	<sys/mman.h>	void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)	Starting address of the mapping	MAP_FAILED, errno set
msync()	<sys/mman.h>	int msync(void *addr, size_t length, int flags)	0	-1, errno set
munmap()	<sys/mman.h>	int munmap(void *addr, size_t length)	0	-1, errno set
close()	<unistd.h>	int close(int fd)	0	-1, errno set

Table 4.2: Purpose of Each System Call in the Project

System call	Purpose in this project	Stage
<code>open()</code>	Opens the selected file with <code>O_RDWR</code> and obtains a file descriptor	File opening
<code>fstat()</code>	Obtains the size and type of the file so that the mapping length can be decided	File information
<code>mmap()</code>	Maps the file into the process address space as a shared, readable and writable region	Memory mapping
(memory access)	Reading and modifying the file through the returned address; no system call is needed for pages that are already present	Mapped access
<code>msync()</code>	Requests that the modified pages be synchronized with the file	Synchronization
<code>munmap()</code>	Removes the mapping from the address space	Unmapping
<code>close()</code>	Releases the file descriptor	File closing

4.4 `mmap()` and Memory Mapping

The `mmap()` call is the central part of the project. In this project it is called with a null address hint, the length equal to the file size, the protection `PROT_READ | PROT_WRITE`, the flag `MAP_SHARED`, the descriptor of the opened file and the offset zero. These parameters mean that the kernel selects the address, the whole file is covered, the memory may be read and written, changes belong to the file, and mapping begins at the start of the file.

The mapping length is rounded up by the kernel to a multiple of the page size. If the file size is not a multiple of the page size, the remaining bytes of the last page are filled with zeros and are not part of the file. Writes to those bytes are not carried to the file, and accessing pages that lie entirely beyond the end of the file raises `SIGBUS`. For this reason the program limits modifications to the file size obtained from `fstat()`.

Common error codes of `mmap()` that are relevant to the project are `EACCES` (the descriptor is not open with the mode required by the requested protection), `EBADF` (invalid descriptor), `EINVAL` (invalid length, flags or offset, for example a zero length), `ENOMEM` (not enough memory or address space) and `ENODEV` (the file system does not support mapping).

4.5 munmap() and Resource Release

The call `munmap()` removes the mapping from the virtual address space of the process. The arguments are the starting address returned by `mmap()` and the same length. After a successful call, an access to the former region is an invalid access and normally results in `SIGSEGV`. The call does not remove the file and does not undo changes already made to the file contents. In this project, `munmap()` is called after `msync()` so that the synchronization request is issued while the mapping is still valid.

4.6 msync() and Memory Synchronization

The call `msync()` is used to request that modifications made through a mapping be synchronized with the underlying file. The project calls `msync(map, length, MS_SYNC)`, where `map` is the address returned by `mmap()`. The start address must be page-aligned, which is satisfied because it is exactly the address returned by `mmap()`. The call returns after the kernel has processed the request, and the return value is checked so that a failure is reported to the user. As explained in Section 2.1.10, the exact behaviour depends on the mapping type and the flags, and the project describes the call as a request for synchronization and not as a general guarantee beyond what the manual page of the call states.

4.7 File Operations — open(), close(), fstat()

The call `open()` is used with the flag `O_RDWR` because a writable shared mapping requires a descriptor that allows both reading and writing. If the file does not exist, if the process has no permission, or if the path names a directory, `open()` returns `-1` and `errno` is set to `ENOENT`, `EACCES` or `EISDIR` respectively. The program prints the corresponding message produced by `strerror()`.

The call `fstat()` fills a `struct stat` with information about the open file. The fields used in the project are `st_size` (the size in bytes), `st_mode` (file type and permission bits, tested with `S_ISREG()` and masked with `0777` for display) and `st_ino` (inode number). Using `fstat()` on the descriptor, rather than `stat()` on the path name, avoids a race in which the file could be replaced between the two calls.

The call `close()` releases the file descriptor. After `close()`, the same number can be returned by a later `open()`. The program checks the return value of `close()` as well, because even this call can fail, for example with `EIO` on some file systems.

4.8 Ubuntu / WSL

Ubuntu provides the Linux kernel interface, the GNU toolchain and the `/proc` pseudo file system that exposes kernel information as files. The file `/proc/<pid>/maps` lists all mapped regions of a process with their address range, permissions, file offset, device, inode and the associated file name. It can be used during a demonstration to show the new region that appears after `mmap()`. When Ubuntu is run under WSL, the same commands and system calls are used, as described in Section 3.3.

4.9 Visual Studio Code

Visual Studio Code is a lightweight source-code editor with syntax highlighting, an integrated terminal and extensions. With the WSL extension, the editor running on Windows can open a folder in the Ubuntu file system

and use the Ubuntu terminal for compiling and running programs. In this project it is used to write the C source file, to view compiler messages and to run the commands described in Chapter 7. Visual Studio Code is a development convenience and is not a part of the program.

4.10 Linux Commands and Utilities

Table 4.3: Linux Commands and Utilities Used

Command	Purpose
gcc	Compile the C program
./program	Run the executable from the current directory
echo, printf, cat	Create small test files and display file contents before and after modification
ls -l	Display file size and permissions
stat file	Display file metadata and compare it with the information printed by the program
chmod	Change file permissions to reproduce permission errors
touch, : > file	Create an empty file for the empty-file test
getconf PAGESIZE	Display the system page size
cat /proc/<pid>/maps	View the memory regions of the running process and locate the mapped file
ps, pgrep	Find the process identifier of the running program
strace (optional)	Trace the system calls issued by the program
xxd or hexdump (optional)	View the file contents in hexadecimal form

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

This chapter describes the design of the Linux Memory-Mapped File Management System. It presents the architecture, the workflow, the data flow, the memory-mapping workflow, the modules and the design of input, output, error handling and resource management. The design follows the logical sequence of the system calls, because the purpose of the project is to make this sequence visible.

5.1 Overall System Architecture

The system is a single user-space process written in C. It receives input from the user, requests services from the Linux kernel through system calls and finally leaves the file in storage with the changes applied. Figure 5.1 shows the layers and the order in which they are used.

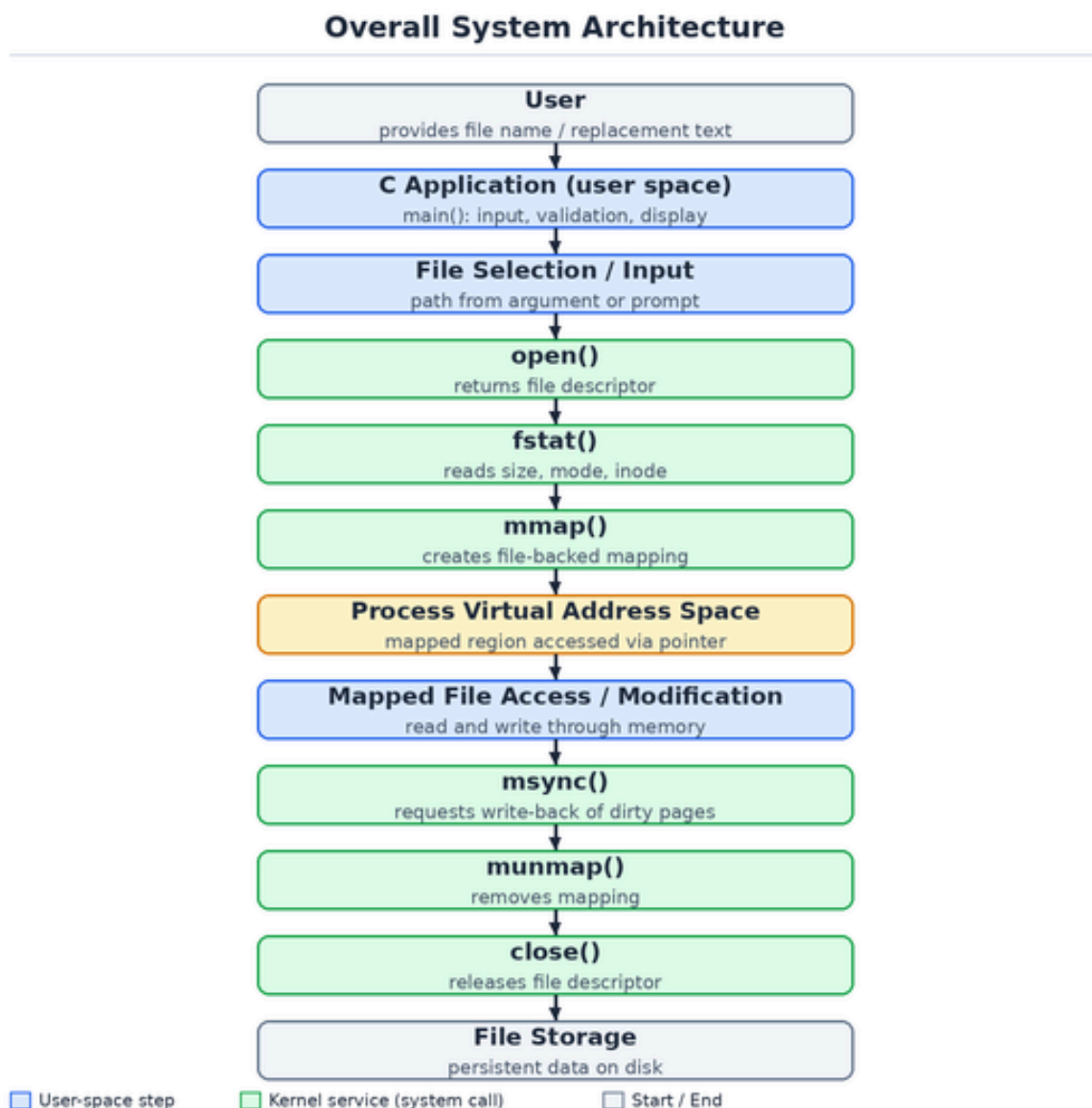


Figure 5.1: Overall System Architecture

The stages of the architecture are explained below.

1. **User.** The user starts the program and supplies the name of the file to be mapped, and later the position and text of a modification. The user is the only external entity of the system.
2. **C Application.** The C program runs in user space. It controls the sequence of operations, validates input, prints information and decides what to do when a call fails.
3. **File Selection / Input.** The file name is taken from the command-line argument or is read from the keyboard when no argument is given. An empty name is rejected before any system call is made.
4. **open().** The kernel locates the file, checks the permissions of the calling process, and returns a file descriptor that identifies the open file in the process.
5. **fstat().** The program asks the kernel for the metadata of the open file. The size obtained here becomes the length of the mapping, and the file type is checked so that only regular files are mapped.
6. **mmap().** The kernel creates a new virtual memory area in the process address space and links it to the file. No file data has necessarily been read yet; pages are loaded on demand.
7. **Process Virtual Address Space.** The returned address lies in this space. The mapped region appears in `/proc/<pid>/maps` as a shared read-write region associated with the file.
8. **Mapped File Access / Modification.** The program reads the contents and then writes the replacement text into the mapped memory with `memcpy()`. The first access to a page can cause a page fault that the kernel resolves by supplying the page.
9. **msync().** The program requests that the modified pages be synchronized with the file, and waits for completion because `MS_SYNC` is used.
10. **munmap().** The mapping is removed from the address space, and the pointer must no longer be used.
11. **close().** The file descriptor is released.
12. **File Storage.** The file in persistent storage contains the modified bytes and can be checked with `cat` or `stat` after the program ends.

5.2 System Workflow

Figure 5.2 shows the workflow of the program. The vertical column shows the normal path. The boxes on the right show what happens when one of the calls fails: the program prints an error message, releases the resources that have been acquired up to that point and terminates with a failure status.

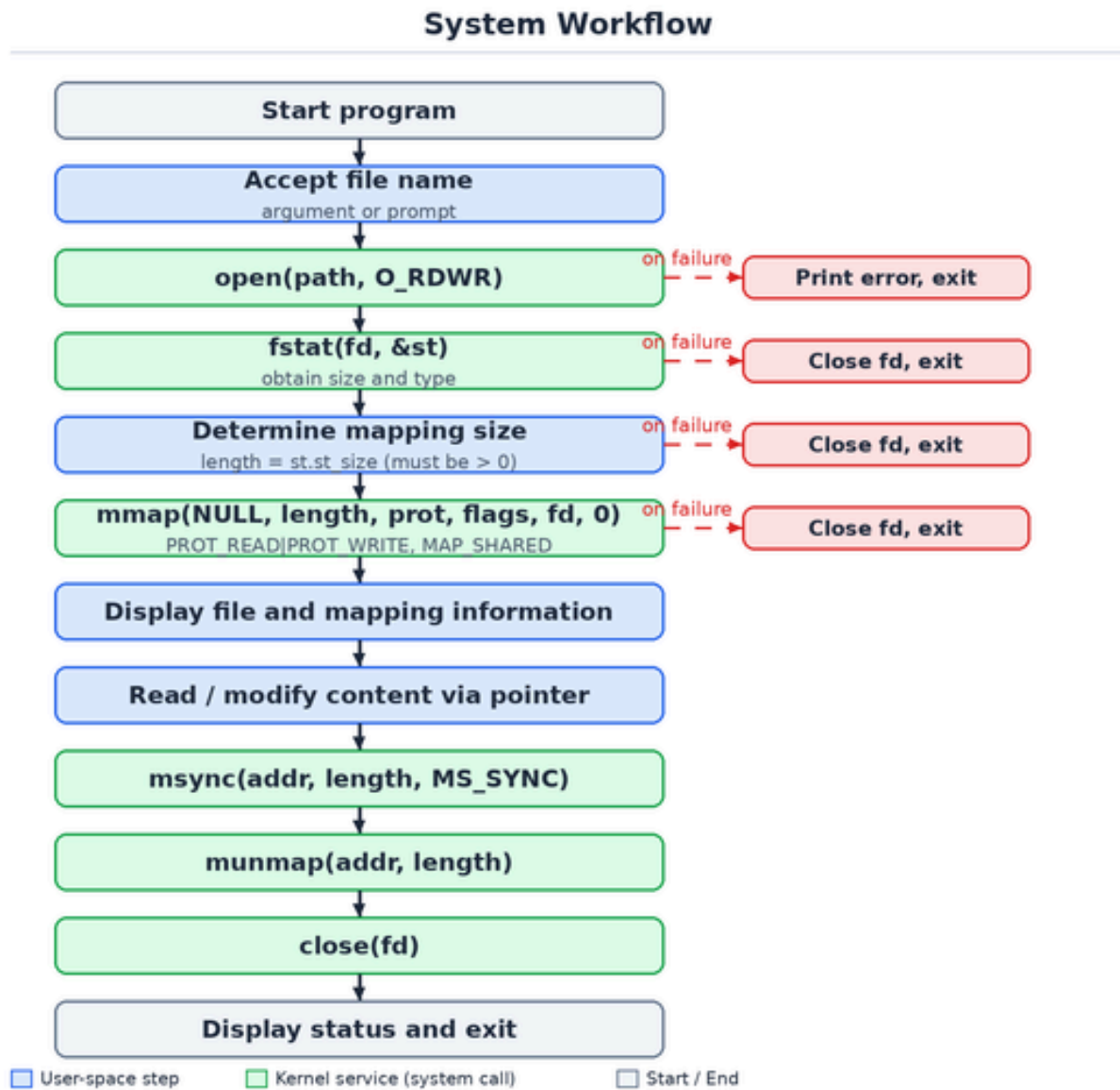


Figure 5.2: System Workflow

The workflow is strictly sequential. A stage is entered only if the previous stage has succeeded, and every stage that follows a successful `open()` closes the descriptor if it has to stop. The release of resources in the reverse order of their acquisition (unmap before close) is shown in the lower part of the figure.

5.3 Data Flow

Figure 5.3 shows the flow of data in the system as a Level 1 data flow diagram. The user supplies the file name, and in the modification step a byte offset and the replacement text. The file name is validated and passed to the file-access process, which returns the descriptor and the file size. These are used by the mapping process to create the mapping and to obtain the address. The read-and-modify process uses the address to display the file contents and to change them. The synchronization, unmapping and closing process finally causes the modified data to be written back to the file on storage, and the final status is displayed to the user.

Data Flow Diagram (Level 1)

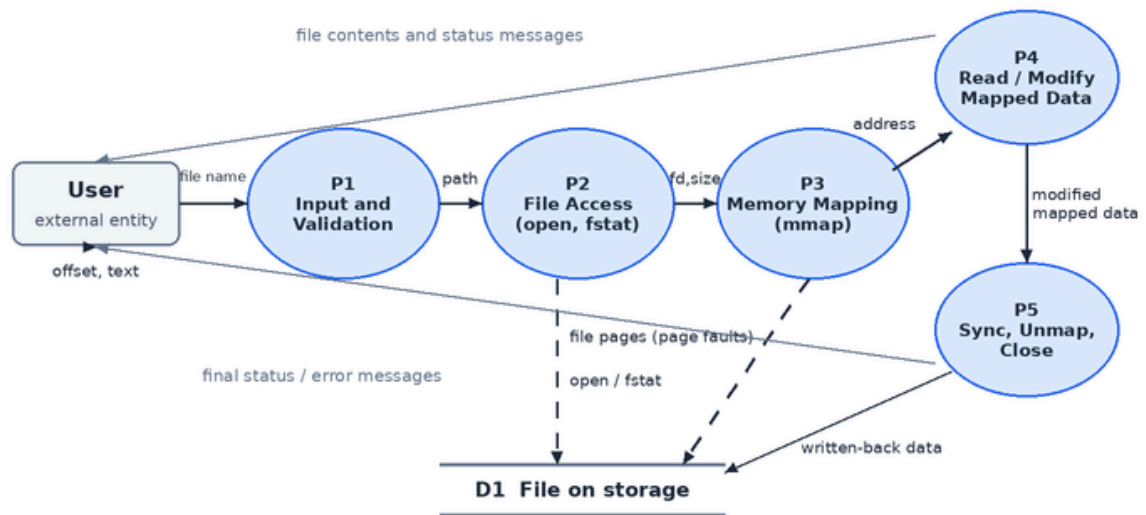


Figure 5.3: Data Flow Diagram (Level 1)

The data store D1 represents the file on persistent storage. Data does not travel between D1 and the program through `read()` or `write()` calls. Instead, file pages are brought into memory by page faults when the mapped region is accessed, and modified pages are written back by the kernel, in particular when `msync()` is called.

5.4 Memory Mapping Workflow

The central part of the system is the creation and use of the mapping. Figure 5.4 shows what happens from the moment the program has a valid descriptor and a known file size until data is available to the program.

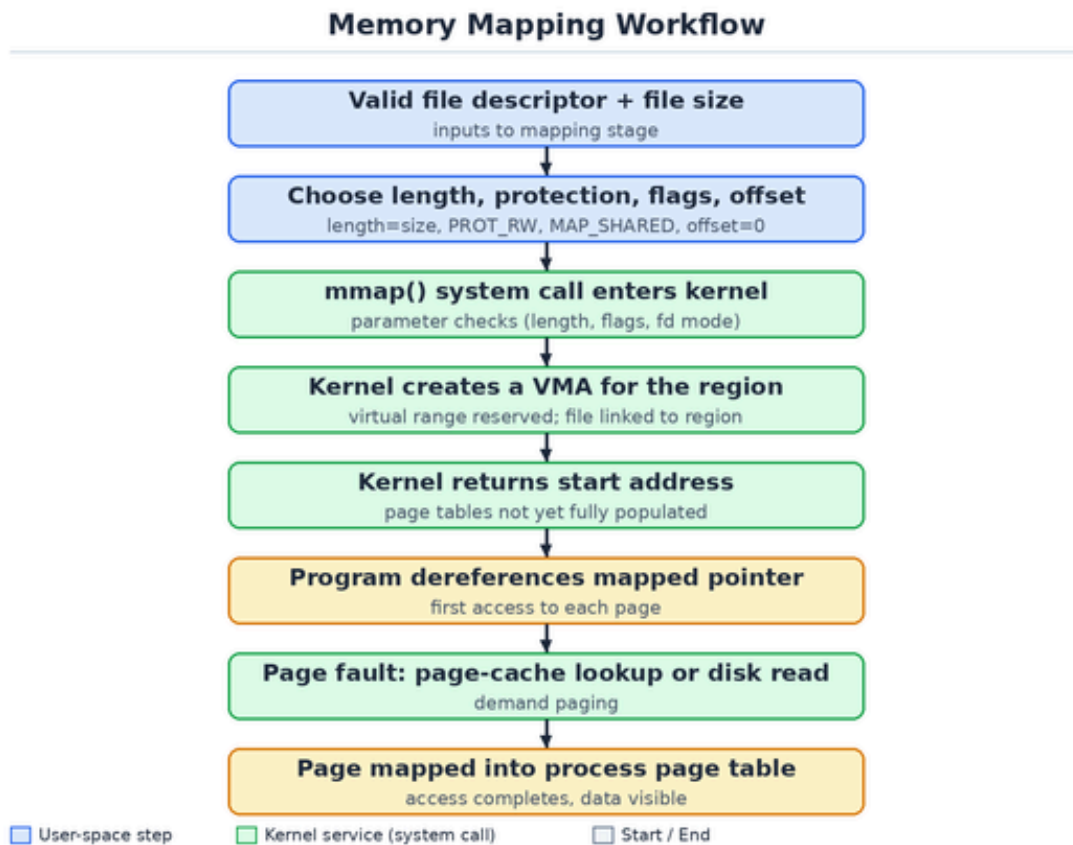


Figure 5.4: Memory Mapping Workflow

When `mmap()` is called, the kernel checks the arguments (for example, that the length is not zero and that the descriptor is open in a mode that allows the requested protection) and creates a virtual memory area that records the address range, the protection, the shared type and the file and offset. The call returns the start address. At this stage the page tables for the region are not necessarily filled. When the program touches a page for the first time, the processor raises a page fault, the kernel looks up the region, obtains the page from the page cache or reads it from the file, installs it in the process page table, and the instruction is executed again successfully. This is demand paging for file-backed memory.

Figure 5.5 shows where the mapped region lies in the virtual address space of the process. The kernel decides the exact location, and the location differs between runs. The region lies between the stack and the heap in the conceptual layout, and it is connected through page tables to pages in the page cache, which in turn correspond to the file on disk.

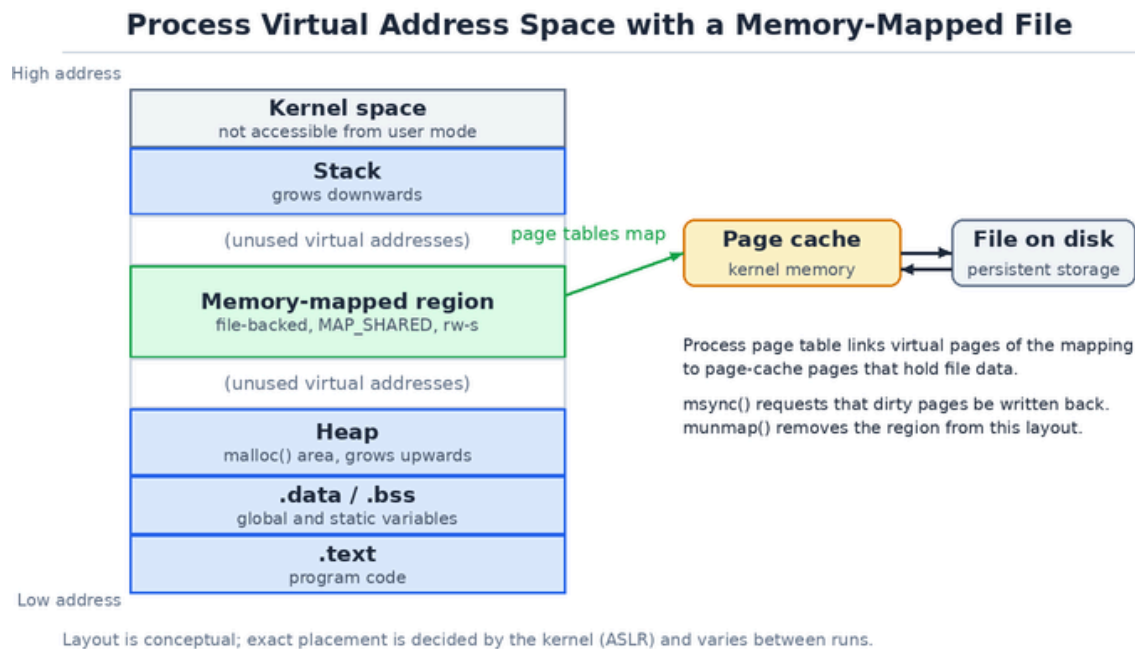


Figure 5.5: Process Virtual Address Space with Memory-Mapped File

The life cycle of a mapping, from the open file to the released descriptor, is shown in Figure 5.6. The states are determined by the system calls that the program makes: the mapping exists from the successful return of `mmap()` until the successful return of `munmap()`, and the descriptor exists from `open()` until `close()`.

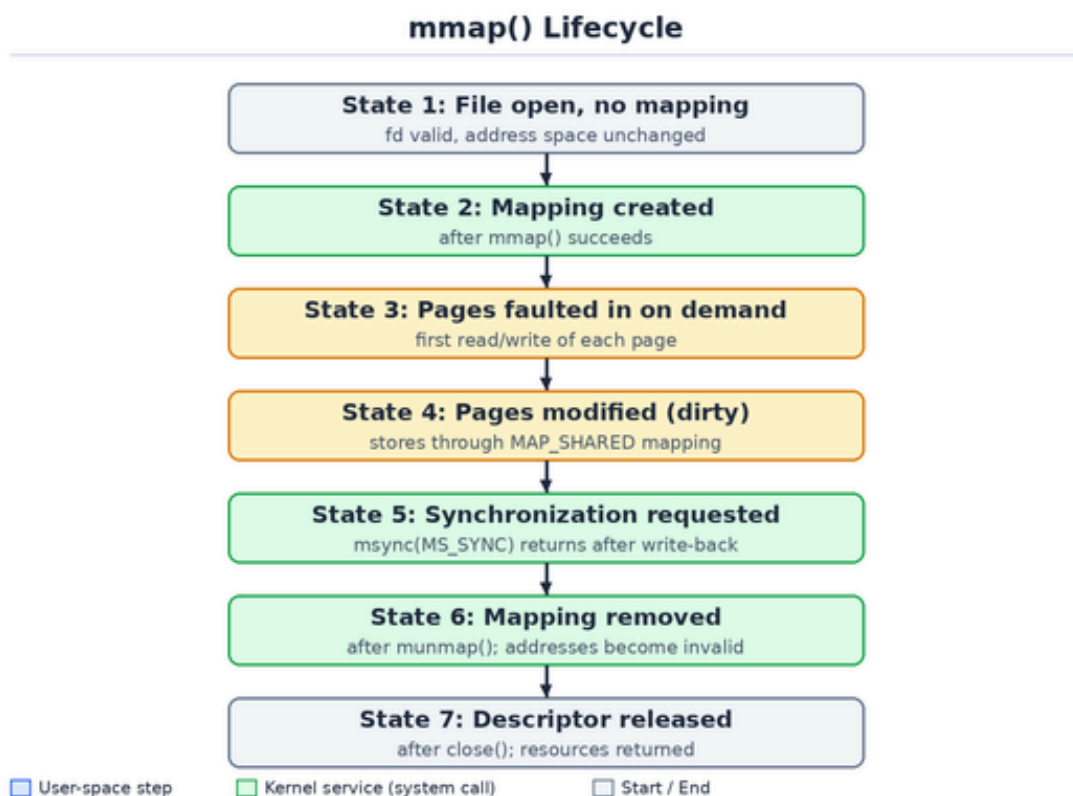


Figure 5.6: mmap() Lifecycle

Figure 5.7 describes the read path and the write path through the system. A read of a mapped address that is not yet present causes a page fault and a lookup in the page cache. A store to a shared mapping modifies the page in the page cache and the page is marked dirty. The kernel writes dirty pages back according to its own policy, and the call `msync()` requests that this be done at a point chosen by the program.

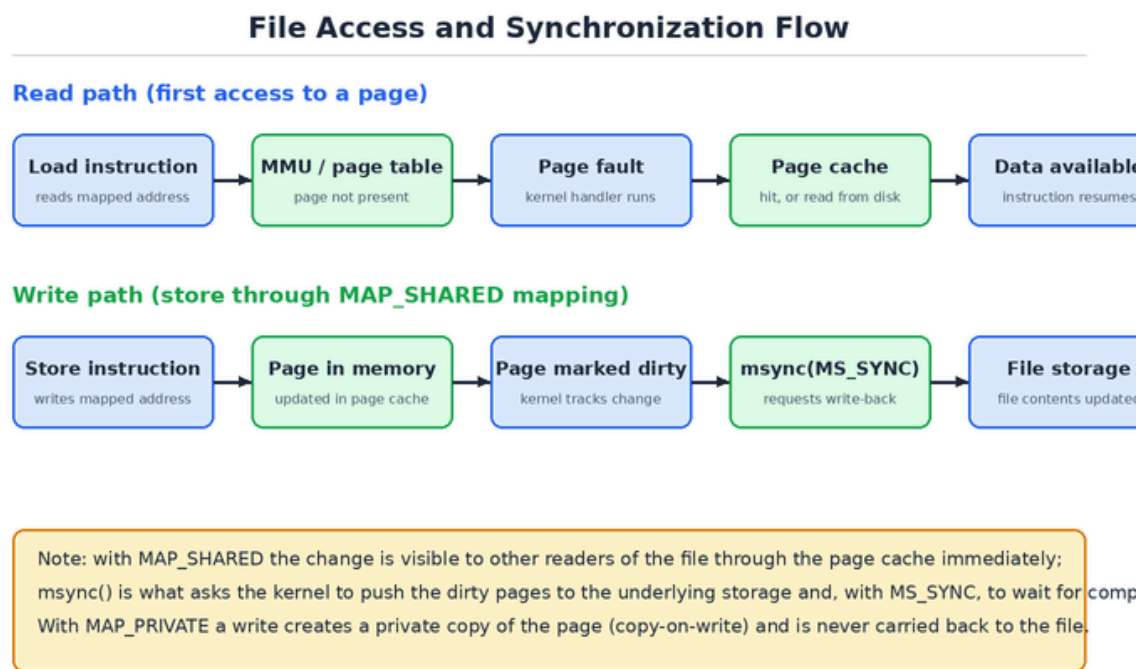


Figure 5.7: File Access and Synchronization Flow

5.5 Module Description

The program is divided into functional modules, each corresponding to one stage of the workflow. In the source code the modules are implemented as clearly separated blocks in the function `main()` and in the helper functions `print_file_info()` and `print_preview()`. Figure 5.8 shows the module architecture and Table 5.1 describes the modules.

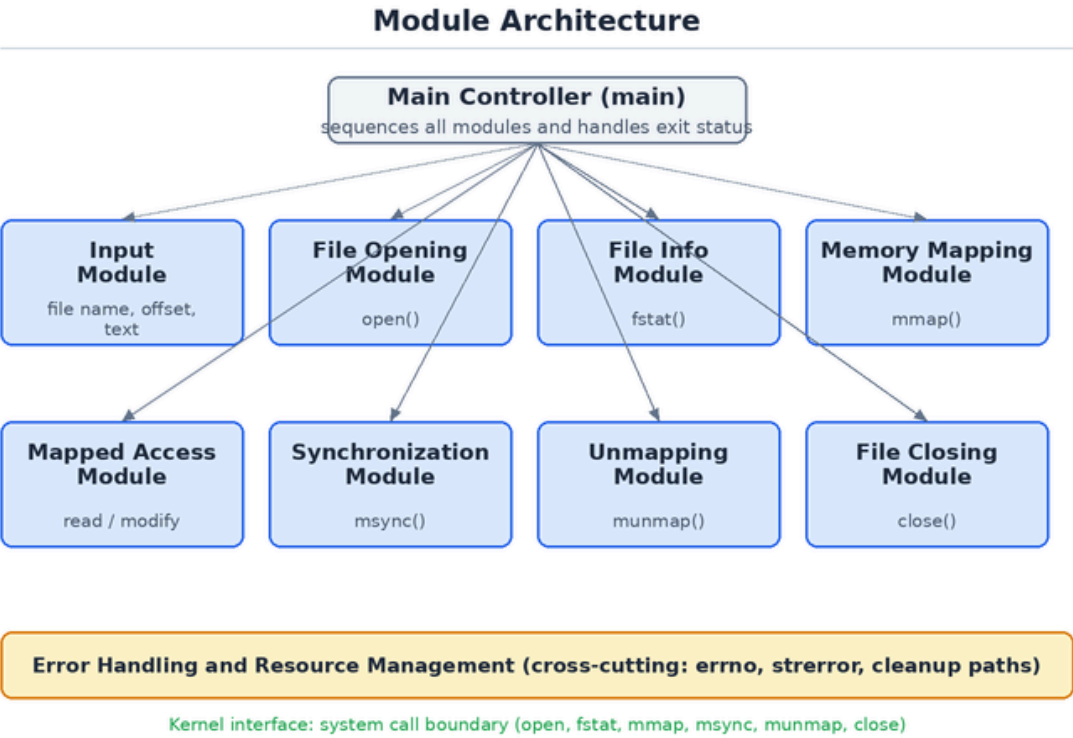


Figure 5.8: Module Architecture

Table 5.1: Project Modules

No.	Module	Responsibility	System call / API	Output of the module
1	Input Module	Obtain the file name from the argument or the prompt; read the offset and the text for modification; validate them	fgets(), strtol()	Validated path, offset and text
2	File Opening Module	Open the file in read-write mode	open()	File descriptor
3	File Information Module	Obtain size, type, permissions and inode; reject non-regular and empty files	fstat(), S_ISREG()	struct stat with metadata
4	Memory Mapping Module	Create a shared read-write mapping over the whole file and display mapping information	mmap(), sysconf()	Start address of the mapping
5	Mapped Memory Access Module	Display the mapped content and modify it through the pointer within the mapped length	Pointer access, memcpy()	Displayed and modified content
6	Memory Synchronization Module	Request synchronization of the modified pages with the file	msync()	Status of the request
7	Memory Unmapping Module	Remove the mapping from the address space	munmap()	Status of the removal
8	File Closing Module	Release the file descriptor	close()	Status of the release
9	Error Handling and Resource Management (cross-cutting)	Check every return value, print messages and release resources on each exit path	errno, strerror()	Error messages and exit status

5.6 Input Design

The program has a small and simple input design. The inputs are text typed on the keyboard or given on the command line. All inputs are read as lines using `fgets()` and are validated before use, because user input should never be trusted. Table 5.2 describes the inputs and the outputs of the program together.

The file name is the main input. It can be given as the first command-line argument (`./[EXECUTABLE_NAME] sample.txt`) or, if it is missing, it is requested at a prompt. The newline character is removed from the entered string, and an empty string is rejected. The byte offset for modification is converted with `strtol()`, and it is accepted only if the conversion succeeds and if the value is between zero and the file size minus one. The replacement text is accepted as one line and is shortened if necessary so that it does not extend beyond the end of the mapped region.

If the submitted source code uses a different input method, for example a menu with numbered choices, this section should be rewritten to describe that method. This report does not claim a menu-driven interface.

5.7 Output Design

The program writes progress and results to standard output and error messages to standard error. Each stage of the workflow prints a short line that states what has been done, so that a person watching the program can follow the sequence of system calls. The main groups of output are the file-opening message with the descriptor value, the file information, the mapping information, the preview of the mapped content, the result of the modification, and the messages that follow `msync()`, `munmap()` and `close()`.

Table 5.2: Input / Output Description

Item	Type	Description / Example
File name	Input	Path of the file to map, for example sample.txt
Byte offset	Input	Position in the mapped region where the modification starts, for example 11
Replacement text	Input	Text copied into the mapped region, limited to the remaining mapped length
File descriptor message	Output	File opened successfully (file descriptor = 3)
File information	Output	Path, size in bytes, permission bits, inode number, regular-file status
Mapping information	Output	Mapped address, length, page size, number of pages, protection, mapping type, process ID
Mapped content preview	Output	First 256 bytes (or the whole file if smaller) read through the mapped address
Modification result	Output	Number of bytes written through the mapping and the offset
Synchronization / unmap / close messages	Output	Confirmation of msync(), munmap() and close()
Error messages	Output (stderr)	Name of the failed call and the text produced by strerror(errno)
Exit status	Output	0 for success, non-zero for failure

5.8 Error Handling Design

Each system call can fail, and each failure is handled at the place where it occurs. The program tests the return value immediately after the call. On failure, the program prints the name of the call and the explanation

obtained from `strerror(errno)` to standard error. If a descriptor has already been opened, it is closed before the program exits. The program terminates with `EXIT_FAILURE` for failures that prevent the mapping and reports non-fatal failures after the mapping (for example an invalid offset) without aborting the remaining cleanup. Table 5.3 summarises the design.

Table 5.3: Error Handling Design

Stage	Possible failure	Typical errno	Response of the program
Input	Empty file name or no input	—	Print message; exit with failure status
<code>open()</code>	File does not exist	<code>ENOENT</code>	Print message; exit
<code>open()</code>	No permission for read-write access	<code>EACCES</code>	Print message; exit
<code>open()</code>	Path is a directory	<code>EISDIR</code>	Print message; exit
<code>fstat()</code>	Metadata cannot be read	<code>EBADF</code> , <code>EIO</code>	Print message; close descriptor; exit
<code>fstat()</code>	File is not a regular file	—	Print message; close descriptor; exit
Size check	File is empty (size 0)	(would be <code>EINVAL</code> in <code>mmap()</code>)	Print message; close descriptor; exit
<code>mmap()</code>	Mapping cannot be created	<code>EACCES</code> , <code>ENOMEM</code> , <code>EINVAL</code> , <code>ENODEV</code>	Print message; close descriptor; exit
Modification	Offset not a number or out of range	—	Print message; skip the modification; continue with cleanup
<code>msync()</code>	Synchronization request fails	<code>EINVAL</code> , <code>ENOMEM</code> , <code>EIO</code>	Print message; continue with unmapping
<code>munmap()</code>	Mapping cannot be removed	<code>EINVAL</code>	Print message; continue with closing
<code>close()</code>	Descriptor cannot be released	<code>EBADF</code> , <code>EIO</code>	Print message

5.9 Resource Management Design

The program acquires two kernel resources: a file descriptor and a memory mapping. The design rule is that resources are released in the reverse order of their acquisition. The mapping, which was created last, is removed first with `munmap()`, and the descriptor, which was acquired first, is released last with `close()`. If the program fails before the mapping is created, only the descriptor has to be released. This rule is applied on every exit path of the program.

The program does not allocate memory dynamically. The mapped region is managed by the kernel, and the buffers used for input are fixed-size arrays on the stack, so there is no memory to be freed with `free()`. The length passed to `munmap()` is the same length that was passed to `mmap()`, and the pointer is not used after unmapping. These rules prevent descriptor leaks, address-space leaks and use-after-unmap errors.

CHAPTER 6 — IMPLEMENTATION

This chapter explains how the design of Chapter 5 is implemented in C. The code in this chapter is a representative implementation of the Linux Memory-Mapped File Management System, written to follow the workflow stated for the project exactly. The representative source file is named `mmap_demo.c` in this report.

Important. The code shown here is a representative implementation prepared for this report. It is not claimed to be the exact text of the source file that was submitted. Before submission, the student must compare this chapter with the actual source file, replace the listings by the real code where they differ, and use the real file name in place of `mmap_demo.c` wherever it appears. The system calls, their order and their parameters are the ones named in the project description.

6.1 Module-wise Implementation

The program is organised so that each stage of the workflow appears as a separate, commented block in `main()`, and two small helper functions handle the display of information. The headers included at the beginning of the file provide the system calls and constants that are listed in Table 3.3. A constant `PREVIEW_BYTES` limits the number of bytes displayed when the mapped content is shown, so that a large file does not flood the terminal.

Header files and constant

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <errno.h>

#include <limits.h>

#include <fcntl.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/stat.h>
```

```
#include <sys/mman.h>
```

```
#define PREVIEW_BYTES 256
```

The local variables of `main()` are the path buffer `path`, a general line buffer `line`, the structure `st` of type `struct stat`, the descriptor `fd`, the pointer `map` that will hold the mapping address, the mapping length and the page size `page_size`. Declaring `map` as `char *` allows byte-wise access to the file contents.

6.2 File Opening Module

The file name is first obtained from the command line or from the keyboard. The program then opens the file in read-write mode. The mode `O_RDWR` is required because the mapping that follows is writable and shared.

File opening (excerpt; "..." marks omitted error handling shown in the full listing)

```
if (argc >= 2) {

    strncpy(path, argv[1], sizeof(path) - 1);

    path[sizeof(path) - 1] = '\0';

} else {

    printf("Enter file name to map: ");

    if (fgets(path, sizeof(path), stdin) == NULL) { ... }

    path[strcspn(path, "\n")] = '\0';

}

fd = open(path, O_RDWR);

if (fd == -1) {

    fprintf(stderr, "Error: open(\"%s\") failed: %s\n",

        path, strerror(errno));

    return EXIT_FAILURE;

}

printf("File opened successfully (file descriptor = %d).\n", fd);
```

The call `open()` returns the lowest unused descriptor, normally 3 in this program, because descriptors 0, 1 and 2 are the standard streams. The value -1 indicates failure, and `errno` explains the cause. Since nothing has been acquired yet, the program can return immediately.

6.3 File Information Module

The program calls `fstat()` on the descriptor to obtain the file metadata, prints it, and then checks the file type and size. The helper function `print_file_info()` performs the printing.

File information and mapping size

```
if (fstat(fd, &st) == -1) {

    fprintf(stderr, "Error: fstat() failed: %s\n", strerror(errno));

    close(fd);

    return EXIT_FAILURE;

}

print_file_info(path, &st);

if (!S_ISREG(st.st_mode)) {

    fprintf(stderr, "Error: only regular files can be mapped ...\n");

    close(fd);

    return EXIT_FAILURE;

}

if (st.st_size == 0) {

    fprintf(stderr, "Error: file is empty; a zero-length mapping is not allowed.\n");

    close(fd);

    return EXIT_FAILURE;

}

length = (size_t)st.st_size;

page_size = sysconf(_SC_PAGESIZE);
```

The structure `struct stat` is filled by the kernel. The field `st_size` has the type `off_t`, which is converted to `size_t` for use as the mapping length. The file is rejected if it is empty because `mmap()` fails with `EINVAL` for a length of zero, and an informative message is more useful than the raw error. After the check, the program has a valid length and, from `sysconf(_SC_PAGESIZE)`, the page size that it uses to compute the number of pages that the mapping spans.

6.4 Memory Mapping Module

Creating the mapping and displaying mapping information

```
map = mmap(NULL, length, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

if (map == MAP_FAILED) {

    fprintf(stderr, "Error: mmap() failed: %s\n", strerror(errno));

    close(fd);

    return EXIT_FAILURE;

}
```

```

printf("Mapped address : %p\n", (void *)map);

printf("Mapped length  : %zu bytes\n", length);

printf("Page size      : %ld bytes\n", page_size);

printf("Pages spanned  : %ld\n", ((long)length + page_size - 1) / page_size);

printf("Protection     : PROT_READ | PROT_WRITE\n");

printf("Mapping type    : MAP_SHARED\n");

```

The parameters of `mmap()` are explained in Table 6.1. The result is compared with `MAP_FAILED` and not with `NULL`, because a failed `mmap()` returns `(void *) -1`. If the call fails, the descriptor is closed before the program exits. The number of pages is calculated by rounding the length up to a whole number of pages, which shows the student that a mapping always occupies whole pages. The program also prints its process identifier and waits for the Enter key, which allows `/proc/<pid>/maps` to be examined from a second terminal while the mapping exists.

Table 6.1: Parameters of `mmap()` as Used in the Project

Parameter	Value used	Meaning
addr	NULL	The kernel chooses the address of the mapping
length	st.st_size	The whole file is mapped; the kernel rounds the region up to whole pages
prot	PROT_READ PROT_WRITE	The region can be read and written
flags	MAP_SHARED	Writes are carried to the file and are visible to other mappings of it
fd	Descriptor returned by <code>open()</code>	Identifies the file to be mapped
offset	0	Mapping starts at the beginning of the file (must be page-aligned)

6.5 Mapped Memory Access Module

After the mapping has been created, the file contents are accessed through the pointer `map`. The helper function `print_preview()` writes the first `PREVIEW_BYTES` bytes (or fewer, for a small file) to standard output directly from the mapped memory.

Reading through the mapping

```
static void print_preview(const char *map, size_t length)
{
    size_t n = (length < PREVIEW_BYTES) ? length : PREVIEW_BYTES;

    printf("\n--- Mapped Content (first %zu bytes) ---\n", n);

    fwrite(map, 1, n, stdout);

    printf("\n--- End of Preview ---\n");
}
```

No `read()` call is used in this function. The bytes are fetched by the processor from the mapped addresses, and the first access to each page causes a page fault that the kernel handles invisibly to the program. The program then modifies the content. The user supplies a byte offset and a replacement text, and the text is copied into the mapped memory with `memcpy()`.

Modifying the mapped region

```
printf("\nEnter byte offset to modify (0 to %zu): ", length - 1);

if (fgets(line, sizeof(line), stdin) != NULL) {

    char *end;

    errno = 0;

    long offset = strtol(line, &end, 10);

    if (end == line || errno != 0 || offset < 0 || (size_t)offset >= length) {

        fprintf(stderr, "Invalid offset. No modification performed.\n");

    } else {

        printf("Enter replacement text: ");

        if (fgets(line, sizeof(line), stdin) != NULL) {

            size_t n = strcspn(line, "\n");

            if (n > length - (size_t)offset)

                n = length - (size_t)offset; /* never write past the mapping */

            memcpy(map + offset, line, n);

            printf("%zu byte(s) written through the mapping at offset %ld.\n",

                n, offset);

        }

    }

}
```

```

    }
}

```

The limit applied to `n` is essential. A mapping cannot extend the file, and writes beyond the end of the file would either be lost in the unused part of the last page or, beyond the last page, would raise `SIGBUS` or `SIGSEGV`. The modification overwrites existing bytes and does not insert text; the size of the file remains unchanged.

6.6 Memory Synchronization Module

Synchronization

```

if (msync(map, length, MS_SYNC) == -1)

    fprintf(stderr, "Error: msync() failed: %s\n", strerror(errno));

else

    printf("msync(MS_SYNC) completed: changes requested to be written to the file.\n");

```

The start address passed to `msync()` is the address returned by `mmap()` and is therefore page-aligned. With `MS_SYNC`, the call returns after the kernel has processed the write-back request for the pages of the region. The message printed by the program is worded as a completed request, which is consistent with the definition of the call. A failure is reported, but the program still proceeds to `unmap` and `close`, so that no resource is left allocated.

6.7 Memory Unmapping Module

Unmapping

```

if (munmap(map, length) == -1)

    fprintf(stderr, "Error: munmap() failed: %s\n", strerror(errno));

else

    printf("munmap() completed: mapping removed from the address space.\n");

```

The same pointer and length that were used for `mmap()` are passed to `munmap()`. After a successful call, the variable `map` still contains the old address, but it is no longer valid, and the program does not use it again.

6.8 File Closing Module

Closing the file

```

if (close(fd) == -1)

    fprintf(stderr, "Error: close() failed: %s\n", strerror(errno));

else

    printf("close() completed: file descriptor released.\n");

printf("\nOperation finished successfully.\n");

return EXIT_SUCCESS;

```

The descriptor is closed last. In general, a mapping remains valid after its descriptor has been closed, but closing after unmapping keeps the order of release the reverse of the order of acquisition, and makes the program simpler to reason about. The final message indicates that all stages completed.

6.9 Error Handling

Error handling in the program follows three rules. First, every system call that can fail has its return value tested immediately: `-1` for `open()`, `fstat()`, `msync()`, `munmap()` and `close()`, and `MAP_FAILED` for `mmap()`. Second, a failure message is written to standard error, and it contains the name of the failed call and the description produced by `strerror(errno)`. Third, the descriptor is closed on every exit path that follows a successful `open()`. In addition to errors reported by the kernel, the program also validates conditions that it can detect itself: an empty file name, a file that is not a regular file, an empty file, and an invalid or out-of-range offset. Section 5.8 summarises the errors and the responses.

The code does not use `exit()` from deep inside helper functions. Failures in `main()` use `return EXIT_FAILURE` after the necessary cleanup, so the flow of control and the release of resources are easy to follow.

6.10 Algorithm

1. Start the program.
2. Obtain the file name from the command-line argument; if it is absent, prompt for it and read it from the keyboard. If the name is empty, display an error and stop.
3. Call `open(path, O_RDWR)`. If it returns `-1`, display the error and stop.
4. Call `fstat(fd, &st)`. If it fails, close the descriptor, display the error and stop. Display the file information.
5. If the file is not a regular file, or if its size is zero, close the descriptor, display the error and stop.
6. Set `length` to the file size and obtain the page size using `sysconf(_SC_PAGESIZE)`.
7. Call `mmap(NULL, length, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)`. If it returns `MAP_FAILED`, close the descriptor, display the error and stop.
8. Display the mapping information: address, length, page size, number of pages, protection, mapping type and process identifier.
9. Display the content of the file by reading it through the mapped address.
10. Read the offset and the replacement text. If the offset is valid, copy the text (limited to the remaining mapped length) into the mapped memory; otherwise display a message and skip the modification.
11. Call `msync(map, length, MS_SYNC)`; display the result.
12. Call `munmap(map, length)`; display the result.
13. Call `close(fd)`; display the result.
14. Display the final success message and stop.

6.11 Pseudocode

Pseudocode of the program

BEGIN

`path` <- command-line argument OR prompt input

 IF `path` is empty THEN print error; `EXIT_FAILURE`

```

fd <- open(path, O_RDWR)

IF fd = -1 THEN print error with strerror(errno); EXIT_FAILURE

print "file opened", fd


IF fstat(fd, st) = -1 THEN close(fd); print error; EXIT_FAILURE

print file information

IF st is not a regular file THEN close(fd); print error; EXIT_FAILURE

IF st.size = 0 THEN close(fd); print error; EXIT_FAILURE

length <- st.size

page_size <- sysconf(_SC_PAGESIZE)


map <- mmap(NULL, length, PROT_READ OR PROT_WRITE, MAP_SHARED, fd, 0)

IF map = MAP_FAILED THEN close(fd); print error; EXIT_FAILURE

print address, length, page size, pages, protection, type


print first min(length, 256) bytes of map

read offset, text

IF offset valid THEN

    n <- min(length(text), length - offset)

    copy n bytes of text to map + offset

ELSE

    print "invalid offset"

END IF


IF msync(map, length, MS_SYNC) = -1 THEN print error ELSE print success

IF munmap(map, length) = -1 THEN print error ELSE print success

IF close(fd) = -1 THEN print error ELSE print success


print "operation finished"

EXIT_SUCCESS

END

```

6.12 Core C Implementation

The complete representative source file is reproduced below. Comments in the file number the steps of the workflow (Step 1 to Step 13) so that the code can be related to the algorithm of Section 6.10 and to the logical workflow of the project.

Listing 6.1: mmap_demo.c (representative implementation)

```
/*
 * mmap_demo.c
 *
 * Linux Memory-Mapped File Management System (representative implementation)
 *
 * Demonstrates: open() -> fstat() -> mmap() -> access/modify -> msync()
 *
 *               -> munmap() -> close()
 *
 * Build : gcc -Wall -Wextra -o mmap_demo mmap_demo.c
 * Run   : ./mmap_demo [file]
 */

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <errno.h>

#include <limits.h>

#include <fcntl.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/stat.h>

#include <sys/mman.h>

#define PREVIEW_BYTES 256

static void print_file_info(const char *path, const struct stat *st)
{
    printf("\n--- File Information (fstat) ---\n");

    printf("Path           : %s\n", path);

    printf("Size           : %lld bytes\n", (long long)st->st_size);
}
```

```

printf("Permissions   : %o (octal)\n", (unsigned)(st->st_mode & 0777));

printf("Inode          : %llu\n", (unsigned long long)st->st_ino);

printf("Regular file   : %s\n", S_ISREG(st->st_mode) ? "yes" : "no");

}

```

```

static void print_preview(const char *map, size_t length)
{
    size_t n = (length < PREVIEW_BYTES) ? length : PREVIEW_BYTES;

    printf("\n--- Mapped Content (first %zu bytes) ---\n", n);

    fwrite(map, 1, n, stdout);

    printf("\n--- End of Preview ---\n");
}

```

```

int main(int argc, char *argv[])
{
    char path[PATH_MAX];

    char line[512];

    struct stat st;

    int fd;

    char *map;

    size_t length;

    long page_size;

    /* Step 1-2: obtain the file name */

    if (argc >= 2) {

        strncpy(path, argv[1], sizeof(path) - 1);

        path[sizeof(path) - 1] = '\0';

    } else {

        printf("Enter file name to map: ");

        if (fgets(path, sizeof(path), stdin) == NULL) {

            fprintf(stderr, "Error: no input received.\n");

            return EXIT_FAILURE;

        }
    }
}

```

```

    path[strcspn(path, "\n")] = '\0';
}

if (path[0] == '\0') {

    fprintf(stderr, "Error: file name must not be empty.\n");

    return EXIT_FAILURE;

}


/* Step 3: open the file for reading and writing */

fd = open(path, O_RDWR);

if (fd == -1) {

    fprintf(stderr, "Error: open(\"%s\") failed: %s\n", path, strerror(errno));

    return EXIT_FAILURE;

}

printf("File opened successfully (file descriptor = %d).\n", fd);


/* Step 4: obtain file metadata */

if (fstat(fd, &st) == -1) {

    fprintf(stderr, "Error: fstat() failed: %s\n", strerror(errno));

    close(fd);

    return EXIT_FAILURE;

}

print_file_info(path, &st);


if (!S_ISREG(st.st_mode)) {

    fprintf(stderr, "Error: only regular files can be mapped by this program.\n");

    close(fd);

    return EXIT_FAILURE;

}


/* Step 5: determine the mapping size */

if (st.st_size == 0) {

    fprintf(stderr, "Error: file is empty; a zero-length mapping is not allowed.\n");

    close(fd);

```

```

    return EXIT_FAILURE;

}

length = (size_t)st.st_size;

page_size = sysconf(_SC_PAGESIZE);


/* Step 6: map the file into the virtual address space */

map = mmap(NULL, length, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

if (map == MAP_FAILED) {

    fprintf(stderr, "Error: mmap() failed: %s\n", strerror(errno));

    close(fd);

    return EXIT_FAILURE;

}


/* Step 7: display mapping information */

printf("\n--- Mapping Information ---\n");

printf("Mapped address : %p\n", (void *)map);

printf("Mapped length   : %zu bytes\n", length);

printf("Page size        : %ld bytes\n", page_size);

printf("Pages spanned    : %ld\n", ((long)length + page_size - 1) / page_size);

printf("Protection       : PROT_READ | PROT_WRITE\n");

printf("Mapping type      : MAP_SHARED\n");

printf("Process ID        : %d\n", (int)getpid());

printf("Press Enter to continue (inspect /proc/%d/maps now if required)...",

      (int)getpid());

fflush(stdout);

(void)fgets(line, sizeof(line), stdin);


/* Step 8: read the file through the mapped memory */

print_preview(map, length);


/* Step 9: modify the mapped region */

printf("\nEnter byte offset to modify (0 to %zu): ", length - 1);

if (fgets(line, sizeof(line), stdin) != NULL) {

```



```

char *end;

errno = 0;

long offset = strtol(line, &end, 10);

if (end == line || errno != 0 || offset < 0 || (size_t)offset >= length) {

    fprintf(stderr, "Invalid offset. No modification performed.\n");

} else {

    printf("Enter replacement text: ");

    if (fgets(line, sizeof(line), stdin) != NULL) {

        size_t n = strcspn(line, "\n");

        if (n > length - (size_t)offset)

            n = length - (size_t)offset; /* never write past the mapping */

        memcpy(map + offset, line, n);

        printf("%zu byte(s) written through the mapping at offset %ld.\n",

            n, offset);

    }

}

}

/* Step 10: synchronize the mapped changes with the file */

if (msync(map, length, MS_SYNC) == -1)

    fprintf(stderr, "Error: msync() failed: %s\n", strerror(errno));

else

    printf("msync(MS_SYNC) completed: changes requested to be written to the file.\n");

/* Step 11: remove the mapping */

if (munmap(map, length) == -1)

    fprintf(stderr, "Error: munmap() failed: %s\n", strerror(errno));

else

    printf("munmap() completed: mapping removed from the address space.\n");

/* Step 12: release the file descriptor */

if (close(fd) == -1)

    fprintf(stderr, "Error: close() failed: %s\n", strerror(errno));

```

```

else

    printf("close() completed: file descriptor released.\n");

/* Step 13: final status */

printf("\nOperation finished successfully.\n");

return EXIT_SUCCESS;

}

```

The program was compiled with `gcc -Wall -Wextra -o mmap_demo mmap_demo.c` using GCC 13.3.0 on Ubuntu 24.04 during the preparation of this report, and compiled without warnings. The student must repeat the compilation of the actual submitted source file and record the result in Chapter 7.

6.13 Linux System Calls / APIs Used

This section collects the parameters, return values and main error codes of the system calls and the supporting functions used in the program.

Table 6.2: Detailed Description of System Calls and Functions

Call	Parameters used in the project	Return value	Main errors considered
<code>open()</code>	<code>path</code> = user file name; <code>flags</code> = <code>O_RDWR</code>	Descriptor ≥ 0 , or -1	<code>ENOENT</code> , <code>EACCES</code> , <code>EISDIR</code>
<code>fstat()</code>	<code>fd</code> = descriptor; <code>buf</code> = <code>&st</code>	0, or -1	<code>EBADF</code> , <code>EIO</code>
<code>mmap()</code>	<code>NULL</code> , <code>length</code> , <code>PROT_READ PROT_WRITE</code> , <code>MAP_SHARED</code> , <code>fd</code> , 0	Address, or <code>MAP_FAILED</code>	<code>EACCES</code> , <code>EBADF</code> , <code>EINVAL</code> , <code>ENOMEM</code> , <code>ENODEV</code>
<code>msync()</code>	<code>addr</code> = <code>map</code> ; <code>length</code> ; <code>flags</code> = <code>MS_SYNC</code>	0, or -1	<code>EINVAL</code> , <code>ENOMEM</code> , <code>EIO</code>
<code>munmap()</code>	<code>addr</code> = <code>map</code> ; <code>length</code>	0, or -1	<code>EINVAL</code>
<code>close()</code>	<code>fd</code> = descriptor	0, or -1	<code>EBADF</code> , <code>EIO</code>
<code>sysconf()</code>	<code>_SC_PAGESIZE</code>	Page size in bytes, or -1	—
<code>getpid()</code>	none	Process identifier	Never fails
<code>strerror()</code>	<code>errno</code>	Pointer to the error text	—
<code>memcpy()</code>	<code>map</code> + offset, text buffer, <code>n</code>	Destination pointer	Undefined behaviour if regions are invalid, hence the length check

The functions `open()`, `fstat()`, `mmap()`, `msync()`, `munmap()` and `close()` are system calls (invoked through the C library), while `sysconf()`, `getpid()`, `strerror()` and `memcpy()` are library functions, of which `getpid()` is also implemented as a system call. The standard output functions are library functions that use `write()` internally for the terminal.

CHAPTER 7 — EXECUTION AND DEMONSTRATION

This chapter explains how the program is compiled and run in Ubuntu or WSL, shows sample input and output, describes how each stage can be demonstrated, and lists the error cases and test cases. The commands use the

placeholder `[SOURCE_FILE].c` and `[EXECUTABLE_NAME]` for the actual file names of the student's submission. In the examples that follow, the representative file name `mmap_demo.c` is used.

About the outputs in this chapter. The sample outputs in Sections 7.3 to 7.9 were produced by compiling and running the representative program of Chapter 6 on Ubuntu 24.04 with GCC 13.3.0 during the preparation of this report. Values such as the mapped address, the inode number and the process identifier differ from machine to machine and from run to run. The student must replace these outputs and add screenshots from the actual submitted program. The test-case table in Section 7.9.2 is left as [TO BE VERIFIED] until the student runs the tests.

7.1 Compilation in Ubuntu / WSL

Open the Ubuntu terminal (or the Ubuntu terminal inside Visual Studio Code), move to the directory that contains the source file, and compile it with GCC.

Compilation commands

```
cd ~/os_project

gcc -Wall -Wextra -g [SOURCE_FILE].c -o [EXECUTABLE_NAME]
```

With the representative file names:

```
gcc -Wall -Wextra -g mmap_demo.c -o mmap_demo
```

If the compilation is successful, GCC prints no output and a new executable file appears in the directory, which can be confirmed with `ls -l`. If there are errors, GCC prints the file name, the line number and an explanation, and the source file must be corrected before running again. The program uses only standard Linux headers, so no additional libraries or linker options are needed.

7.2 Execution Commands

A test file is first created, and the program is then run with the file name as the argument. If the argument is omitted, the program asks for the file name.

Execution commands

```
printf 'Operating Systems project\nMemory mapped file demonstration.\nLine three of sample file.\n' > sample.txt
```

```
cat sample.txt
```

```
ls -l sample.txt
```

```
./[EXECUTABLE_NAME] sample.txt          # file name as argument
```

```
./[EXECUTABLE_NAME]                    # file name requested at the prompt
```

After the program finishes, the file is displayed again with `cat sample.txt` to confirm the modification. The page size and kernel version can be shown with `getconf PAGESIZE` and `uname -r`.

7.3 Sample Input

The test file `sample.txt` used in the demonstration contains three lines of text and has a size of 87 bytes.

Contents of sample.txt before the run

```
Operating Systems project
```

```
Memory mapped file demonstration.
```

```
Line three of sample file.
```

The inputs given to the program during the run are the file name `sample.txt`, the Enter key at the pause after the mapping information, the byte offset 11 and the replacement text `MAPPED_REPLACEMENT`.

7.4 Sample Output

The following listing shows the complete output of one run. The lines that start after a prompt contain the values typed by the user.

Sample output of the representative program

```
$ ./mmap_demo sample.txt
```

```
File opened successfully (file descriptor = 3).
```

```
--- File Information (fstat) ---
```

```
Path           : sample.txt
```

```
Size           : 87 bytes
```

```
Permissions    : 644 (octal)
```

```
Inode          : 573460
```

```
Regular file   : yes
```

```
--- Mapping Information ---
```

```
Mapped address : 0x7f3fe8dcf000
```

```
Mapped length  : 87 bytes
```

```
Page size      : 4096 bytes
```

```
Pages spanned  : 1
```

```
Protection     : PROT_READ | PROT_WRITE
```

```
Mapping type    : MAP_SHARED
```

```
Process ID     : 121
```

```
Press Enter to continue (inspect /proc/121/maps now if required)...<Enter>
```

```
--- Mapped Content (first 87 bytes) ---
```

Operating Systems project

Memory mapped file demonstration.

Line three of sample file.

--- End of Preview ---

Enter byte offset to modify (0 to 86): 11

Enter replacement text: MAPPED_REPLACEMENT

18 byte(s) written through the mapping at offset 11.

msync(MS_SYNC) completed: changes requested to be written to the file.

munmap() completed: mapping removed from the address space.

close() completed: file descriptor released.

Operation finished successfully.

The file contents after the run, displayed with `cat sample.txt`, are shown below. Eighteen bytes starting at offset 11 have been overwritten, and the file size remains 87 bytes.

Contents of sample.txt after the run

Operating SMAPPED_REPLACEMENTory mapped file demonstration.

Line three of sample file.

7.5 Demonstration of mmap()

The demonstration of `mmap()` has three parts. First, the output of the stage "Mapping Information" is shown: the program prints the virtual address returned by `mmap()`, the length of the mapping, the page size and the number of pages. For the 87-byte test file, the number of pages is 1, which shows that the mapping occupies a whole page although the file is much smaller.

Second, while the program waits at the "Press Enter" prompt, a second terminal is opened and the memory map of the process is displayed with the process identifier printed by the program.

Viewing the mapping of the running process

```
cat /proc/121/maps | grep sample.txt
```

A line of the following form appears. The address range (here exactly one page long), the permission field `rw-s`, the offset, the device and inode numbers and the file name are shown. The letter `s` indicates a shared mapping, and the permission letters `rw` show the read and write protection requested.

Format of a `/proc/<pid>/maps` line for the mapped file (values vary)

```
7fe266a36000-7fe266a37000 rw-s 00000000 fe:00 573460 /home/user/os_project/sample.txt
```

Third, the inode number printed by the program is compared with the output of `stat sample.txt`, which confirms that the mapping and the command refer to the same file.

7.6 Demonstration of File Modification

The modification is demonstrated by showing the file contents with `cat` before the run, giving an offset and a replacement text during the run, and showing the contents again after the run. Between the two displays, no `write()` call is made by the program for the data; the change is made by a `memcpy()` into the mapped memory. The student should point out that the line "18 byte(s) written through the mapping" is produced after an ordinary memory copy and not after a file-writing call.

The student can also show the limit check. If the replacement text is longer than the remaining mapped length, the text is shortened and the file size is unchanged, which can be verified with `ls -l`.

7.7 Demonstration of `msync()`

The call `msync()` is demonstrated through the message "msync(MS_SYNC) completed", which appears after the modification and before the unmapping. The student should explain that the modification was made in memory, that `msync()` with `MS_SYNC` requests that the modified pages be written to the file and returns after the request has been processed, and that the file shows the new contents when it is inspected afterwards. The student should also be able to explain that on Linux the new contents of a shared mapping can be seen by other readers of the file through the page cache even before `msync()`, and that `msync()` is the request that concerns the write-back to storage.

If `strace` is installed, the following command lists the relevant system calls, including `msync`. In the output of `strace`, `open()` normally appears as `openat`, because the C library uses that system call internally.

Tracing the system calls (optional)

```
strace -e trace=openat,fstat,newfstatat,mmap,msync,munmap,close ./[EXECUTABLE_NAME] sample.txt
```

7.8 Demonstration of `munmap()`

The message "munmap() completed: mapping removed from the address space" shows that the region has been removed. If the process could be inspected again after this message, the line for `sample.txt` would no longer appear in `/proc/<pid>/maps`. Because the program ends immediately afterwards, a debugger such as `gdb` or a short pause added to the code is needed for this check, and it is optional. The student should explain that any access to the old address after `munmap()` would be an invalid access and would raise `SIGSEGV`, and that the file itself and its data are not affected by `munmap()`.

After the message "close() completed", the file descriptor has been released. The student can state that no resource of the program remains allocated at the end of the run.

7.9 Error Cases

7.9.1 Error Case Descriptions

The following error situations were reproduced with the representative program. In each case the program terminates with a failure status after printing an explanatory message on standard error.

Error cases and the messages produced

```
$ ./mmap_demo nofile.txt
```

```
Error: open("nofile.txt") failed: No such file or directory
```

```
$ ./mmap_demo empty.txt
```

```
File opened successfully (file descriptor = 3).
```

```
...
```

```
Error: file is empty; a zero-length mapping is not allowed.
```

```
$ ./mmap_demo /tmp
```

```
Error: open("/tmp") failed: Is a directory
```

```
$ ./mmap_demo /dev/null
```

```
...
```

```
Regular file : no
```

```
Error: only regular files can be mapped by this program.
```

```
$ chmod 444 ro.txt ; (run as an ordinary user without write permission)
```

```
$ ./mmap_demo ro.txt
```

```
Error: open("ro.txt") failed: Permission denied
```

```
$ (ulimit -v 1000000; ./mmap_demo big.bin)      # 2 GB file, 1 GB address-space limit
```

```
...
```

```
Error: mmap() failed: Cannot allocate memory
```

```
$ printf '\nabc\n' | ./mmap_demo sample.txt      # invalid offset
```

```
...
```

```
Invalid offset. No modification performed.
```

The permission test only fails for a user who does not have write access. When the program is run as the superuser (root), the file permission bits are not enforced in the same way, so the permission test must be run as an ordinary user. The "mmap() failed" test uses `ulimit -v` to reduce the address space that is available to the process, which is a way of causing `ENOMEM` without damaging the system.

7.9.2 Test Cases

Table 7.1 lists the test cases prepared for the project. The column "Expected Result" is based on the design of Chapter 5 and on the behaviour of the system calls. The column "Actual Result" is marked [TO BE VERIFIED] because the student must run the tests on the submitted program and record the results, preferably with screenshots. Each case should then be marked Pass or Fail.

Table 7.1: Test Cases

ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC-01	Valid file opening	sample.txt (existing, readable and writable)	open() succeeds; descriptor value (3) is printed	[TO BE VERIFIED]	Pending
TC-02	Invalid file path	nofile.txt (does not exist)	Message "No such file or directory"; failure status; no mapping created	[TO BE VERIFIED]	Pending
TC-03	Empty file handling	empty.txt (size 0)	Message that an empty file cannot be mapped; descriptor closed; failure status	[TO BE VERIFIED]	Pending
TC-04	File metadata retrieval	sample.txt	Size, permissions and inode printed match ls -l and stat	[TO BE VERIFIED]	Pending
TC-05	Successful mmap()	sample.txt	Mapped address, length 87, page size 4096, pages 1 are displayed	[TO BE VERIFIED]	Pending
TC-06	mmap() failure	2 GB file with address-space limit set by ulimit -v	Message "Cannot allocate memory"; descriptor closed; failure status	[TO BE VERIFIED]	Pending
TC-07	Reading mapped contents	sample.txt	Preview shows the same text as cat sample.txt	[TO BE VERIFIED]	Pending
TC-08	Modifying mapped contents	Offset 11; text MAPPED_REPLACE MENT	18 bytes written; message printed; cat shows modified text; size unchanged	[TO BE VERIFIED]	Pending
TC-09	Successful msync()	After TC-08	Message that msync() completed; modified text present in the file afterwards	[TO BE VERIFIED]	Pending
TC-10	munmap() execution	After TC-09	Message that the mapping was removed; no error	[TO BE VERIFIED]	Pending
TC-11	close() execution	After TC-10	Message that the descriptor was released; exit status 0 (echo \$?)	[TO BE VERIFIED]	Pending

TC-12	Permission-related failure	ro.txt with mode 444, run as an ordinary user	Message "Permission denied"; failure status	[TO BE VERIFIED]	Pending
TC-13	Invalid input (offset)	Offset abc, or 500 for an 87-byte file	Message "Invalid offset"; file unchanged; cleanup (msync, munmap, close) still executed	[TO BE VERIFIED]	Pending
TC-14	Invalid input (empty file name)	Press Enter at the prompt	Message that the file name must not be empty; failure status	[TO BE VERIFIED]	Pending
TC-15	Non-regular file	/dev/null	Message that only regular files are supported; descriptor closed	[TO BE VERIFIED]	Pending
TC-16	Replacement text longer than remaining length	Offset 80; text of 30 characters	Text shortened to 7 bytes; file size unchanged; no crash	[TO BE VERIFIED]	Pending

7.10 Screenshots to be Included

The following screenshots must be captured by the student from the actual submitted program and inserted at the places indicated. No screenshot has been fabricated for this report.

Table 7.2: Screenshots to be Included

No.	Screenshot	Related Section
1	Source file open in Visual Studio Code (WSL or Ubuntu)	6.12
2	Compilation command and successful compilation in the terminal	7.1
3	Creation and contents of the test file (cat and ls -l)	7.2, 7.3
4	Program start, file opened message and file information	7.4
5	Mapping information (address, length, page size, pages)	7.5
6	/proc/<pid>/maps showing the mapped file with rw-s	7.5
7	Preview of the mapped content	7.4
8	Modification step with offset and replacement text	7.6
9	msync(), munmap() and close() messages and final success message	7.7, 7.8
10	Contents of the file after the run (cat)	7.6
11	Error cases: invalid file, empty file, permission denied	7.9.1
12	Completed test-case table with actual results	7.9.2

[Insert actual screenshots here: compilation, execution, /proc/<pid>/maps, modification and error cases]

7.11 Demonstration of OS Concepts

During the project demonstration, the student should not only run the program but also relate each visible output to an operating-system concept. A suggested sequence for the demonstration is given below.

1. **Introduce the project** as a user-space Linux program that uses system calls to demonstrate memory management. State clearly that it is not a kernel module and not an operating system.
2. **Show the source file** and point out the six system calls in their order: `open()`, `fstat()`, `mmap()`, `msync()`, `munmap()`, `close()`.
3. **Compile and run** the program and show the file descriptor value. Explain that the descriptor is an index in the process's table of open files, usually starting at 3.
4. **Show the file information** and compare it with `stat`. Explain that `fstat()` is a system call that asks the kernel for metadata and that the size becomes the mapping length.
5. **Show the mapping information.** Explain that the address is a virtual address, that it differs between runs because of address-space layout randomization, and that the mapping occupies whole pages of 4096 bytes.
6. **Show `/proc/<pid>/maps`**. Point out the new region with `rw-s` and the file name, which proves that the file is part of the process address space.
7. **Explain demand paging.** The file has not been copied into memory by `mmap()`; pages are loaded when first accessed through a page fault, and the pages come from the page cache.
8. **Modify the file through memory** and show the unchanged size and changed content. Explain `MAP_SHARED` and contrast it with `MAP_PRIVATE`, where the change would remain private to the process.
9. **Explain `msync()`** as the request to synchronize the mapped modifications with the file, and distinguish volatile memory from persistent storage.
10. **Explain `munmap()` and `close()`** and the principle of releasing resources in the reverse order of acquisition.
11. **Show error handling** with an invalid file, an empty file and a permission failure, and explain the role of `errno` and `strerror()`.
12. **Discuss the limitations** (fixed mapping length, no file growth, no benchmark data) honestly, and describe the future enhancements of Section 8.7.

Table 8.2 in the next chapter lists the operating-system concepts and the part of the program where each of them can be seen.

CHAPTER 8 — RESULTS, CONCLUSION AND FUTURE SCOPE

8.1 Summary of the Project

The project titled Linux Memory-Mapped File Management System was designed and implemented as a user-space program in C for the Linux operating system. The program follows the complete life cycle of a memory-mapped file. It opens a file with `open()`, obtains its metadata with `fstat()`, determines the mapping size, maps the file into the process virtual address space with `mmap()`, displays the file and mapping information, gives access to the content through a memory address, modifies the content through the mapped region, requests synchronization with `msync()`, removes the mapping with `munmap()` and releases the descriptor with `close()`. Every stage checks for failure and reports an understandable message.

The project is not a text editor and not a kernel component. Its value is educational: it makes the cooperation between the process address space, the page cache, the file system and the system-call interface visible in a

small program that can be explained completely during a viva.

8.2 Objectives Achieved

Table 8.1 relates each objective stated in Section 1.4 to the part of the project in which it is addressed. The results of the tests of Table 7.1 must be verified and entered by the student before the final statement of achievement is made in the submitted report.

Table 8.1: Objectives and How They Are Addressed

Objective	How it is addressed	Where
Study the relationship between address space, file-backed memory and the file	Concept study, address-space diagram, /proc/<pid>/maps demonstration	Sections 2.1, 5.4, 7.5
Implement the complete life cycle of a mapping	Eight-stage workflow implemented in main()	Sections 5.2, 6.2–6.8
Use the six system calls with checked return values	Each call is followed by a test of its result	Sections 6.2–6.9, 6.13
Show that a file can be read and modified as memory	Preview with fwrite() from the mapping; modification with memcpy()	Sections 6.5, 7.6
Explain shared and private mappings and protection flags	Theory and explanation of MAP_SHARED with PROT_READ PROT_WRITE	Sections 2.1.9, 6.4
Demonstrate safe resource management	Release in reverse order of acquisition; cleanup on every exit path	Sections 5.9, 6.9
Provide a documented example for viva and laboratory use	Complete report, demonstration plan and viva points	Chapter 7, Viva points

8.3 Results Achieved

The following results were obtained from the design and from the compilation and trial run of the representative implementation. The results of the formal test cases remain to be confirmed by the student.

- The program compiles with `gcc -Wall -Wextra` without warnings in the environment used for the preparation of this report.
- A shared read-write mapping of a small text file (87 bytes) was created, and the information printed by the program (one page of 4096 bytes spanned) was consistent with the file size and the system page size.
- The mapped region was visible in `/proc/<pid>/maps` with the permission string `rw-s`, which confirms a shared writable file-backed mapping.
- A modification made through the mapped memory changed the content of the file, and the file size remained unchanged.
- The error cases (missing file, empty file, directory, non-regular file, permission denied, address-space limit and invalid offset) produced meaningful messages and a failure status without abnormal termination.

No quantitative benchmark results were collected in the current implementation; therefore, performance claims are discussed conceptually rather than numerically (see Section 2.2.3).

8.4 Operating System Concepts Demonstrated

Table 8.2: Operating System Concepts Demonstrated

OS concept	Where it appears in the project	How it can be shown
System call / user-kernel boundary	open(), fstat(), mmap(), msync(), munmap(), close()	strace output; source code
Process and process address space	The mapped region is added to the address space of the running process	/proc/<pid>/maps
Virtual address vs physical address	The pointer returned by mmap() is a virtual address that changes between runs	Printed address in two runs
Virtual memory and memory pages	Mapping occupies whole pages; pages spanned is printed	Page size and pages spanned output
Demand paging and page faults	Pages of the file are loaded on first access to the mapped memory	Explanation of the preview step; optional /usr/bin/time -v or perf
File-backed memory and the page cache	Mapped pages correspond to cached pages of the file	Modification visible in the file after msync()
Shared vs private mapping	MAP_SHARED is used so that changes reach the file	Source code; the s in rw-s
Protection flags	PROT_READ PROT_WRITE; O_RDWR needed for writable shared mapping	Source code; permission string rw-s
File descriptor vs memory address	fd value 3 compared with the mapping address	Program output
Memory synchronization and persistent storage	msync(MS_SYNC) requests write-back to the file	Message and file content after the run
Kernel involvement	Kernel creates the mapping, handles faults and writes back pages	Explanation with Figures 5.4 to 5.7
Error handling	errno and strerror() messages for every failure	Error-case runs
Resource management	munmap() before close(); descriptor closed on all exit paths	Source code; absence of leaks

--	--	--

8.5 Skills and Learning Outcomes

By completing the project, the student has gained the following skills and understanding.

- Practical knowledge of the Linux system-call interface and of the way C library functions wrap system calls.
- Understanding of virtual memory, the process address space, paging, page faults and the page cache, and how they relate to file access.
- Ability to use `mmap()`, `msync()` and `munmap()` correctly, including the choice of protection and sharing flags.
- Habit of checking every return value, interpreting `errno` and releasing resources in the correct order.
- Ability to use Linux tools such as `gcc`, `stat`, `chmod`, `/proc/<pid>/maps` and `strace` for development and verification.
- Experience with the Ubuntu and WSL development environment, and with writing technical documentation for a system-programming project.

8.6 Limitations

The project has the following limitations, which are stated honestly so that the report is not read as claiming more than the program does.

- The program is a user-space demonstration. It does not implement or modify any part of the kernel, and it does not implement a virtual-memory subsystem; those functions are carried out by Linux.
- Only one file is mapped at a time, and the mapping covers the whole file starting at offset zero.
- The file size cannot be increased through the mapping, and an empty file cannot be mapped.
- Modification overwrites existing bytes and does not insert or delete text.
- No quantitative benchmark results were collected; the comparison with `read()` and `write()` is conceptual only.
- The behaviour of `msync()` and of the page cache is described according to the Linux manual pages, and the program does not measure when data actually reaches the storage device.
- There is no protection against simultaneous modification of the same file by other processes.
- The interface is a simple command-line interface.

8.7 Future Enhancements

The following enhancements are possible extensions of the project. None of them is implemented in the current version.

- A better user interface, for example a menu-driven interface or a text-based interface with a mapped-content viewer.
- Support for several files and several mapping regions at the same time, with separate offsets and lengths.
- Monitoring of the file size and detection of changes to the file by other processes while it is mapped.

- A mapping status display that reads and formats `/proc/self/maps` to show the region, its permissions and its size.
- Visualization of access permissions and of the difference between `MAP_SHARED` and `MAP_PRIVATE` behaviour.
- A mapping log that records each operation with a time stamp.
- A performance comparison with `read()` and `write()` for different file sizes and access patterns, using repeated measurements, so that numerical evidence can replace the conceptual discussion.
- More detailed error reporting, including decoding of `errno` values and recommended actions.
- Multi-threaded access to the mapped region with synchronization (for example mutexes), and demonstration of multi-process sharing of a mapped file.
- Advanced demonstrations such as anonymous mappings, `mprotect()`, `madvise()`, `mremap()` and `ftruncate()` for growing a file before mapping.

8.8 Conclusion

The Linux Memory-Mapped File Management System demonstrates, in a compact and understandable manner, how a file on persistent storage can become part of the virtual address space of a process and how it can be read and modified with ordinary memory operations. The project shows the role of each system call, from `open()` and `fstat()`, which prepare the mapping, to `mmap()`, which creates it, to `msync()`, `munmap()` and `close()`, which synchronize, remove and release it. The discussion of pages, page faults, demand paging, the page cache and shared mappings explains what the kernel does on behalf of the program.

The project does not claim to be a kernel, a virtual-memory manager or a performance optimization. It is a user-space Linux system-programming exercise that turns abstract operating-system concepts into observable behaviour. It therefore serves its purpose as a practical Operating Systems project and gives a base for further study of memory management, inter-process communication and file systems.

CHAPTER 9 — REFERENCES

Check edition, year and publisher against the copy of each book that you actually used. Add only those sources that you have consulted. The list below contains genuine, standard sources only; no URL, DOI or paper has been invented.

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
2. M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.
3. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2015.
4. W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed., Addison-Wesley, 2013.
5. R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, 2016.
6. R. Love, *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010.
7. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988.

9.2 Research Papers

1. P. J. Denning, "Virtual Memory," *ACM Computing Surveys*, vol. 2, no. 3, pp. 153–189, 1970.

2. A. Crotty, V. Leis and A. Pavlo, "Are You Sure You Want to Use MMAP in Your Database Management System?," *Conference on Innovative Data Systems Research (CIDR)*, 2022.

These two papers are given as background reading on virtual memory and on the practical issues of memory-mapped I/O. The student should include them only if they have been consulted.

9.3 Linux / GNU Documentation

1. Linux man-pages project: `mmap(2)`, `munmap(2)`, `msync(2)`, `open(2)`, `close(2)`, `fstat(2)`, `errno(3)`, `sysconf(3)`, `strerror(3)` and `proc(5)`.
2. Free Software Foundation, *Using the GNU Compiler Collection (GCC)*, GNU documentation.
3. Free Software Foundation, *The GNU C Library Reference Manual*.
4. IEEE and The Open Group, *POSIX.1 (IEEE Std 1003.1) / Single UNIX Specification*, specification of `mmap()`, `munmap()`, `msync()`, `open()`, `close()` and `fstat()`.
5. The Linux kernel documentation, memory management section.

9.4 Online Resources

1. Linux man-pages online: <https://man7.org/linux/man-pages/>
2. GCC online documentation: <https://gcc.gnu.org/onlinedocs/>
3. GNU C Library manual: <https://www.gnu.org/software/libc/manual/>
4. POSIX specifications (The Open Group): <https://pubs.opengroup.org/onlinepubs/9699919799/>
5. The Linux Kernel documentation: <https://www.kernel.org/doc/html/latest/>
6. Microsoft documentation for Windows Subsystem for Linux: <https://learn.microsoft.com/windows/wsl/>
7. Ubuntu official website: <https://ubuntu.com/>

APPENDIX A — DIAGRAM SPECIFICATIONS

The diagrams of Chapter 5 were drawn for this report. The specifications below list the boxes (nodes) and arrows of each diagram so that the diagrams can be redrawn in a tool such as draw.io, PowerPoint or Lucidchart if the student wishes to change their appearance.

Table 9.1: Diagram Specifications

Figure	Title	Boxes / nodes	Arrows	Explanation
5.1	Overall System Architecture	User; C Application; File Selection / Input; open(); fstat(); mmap(); Process Virtual Address Space; Mapped File Access / Modification; msync(); munmap(); close(); File Storage	Each box to the next one, from top to bottom	Layers and order of use of system calls from the user to the file
5.2	System Workflow	Start; Accept file name; open(); fstat(); Determine mapping size; mmap(); Display information; Read / modify; msync(); munmap(); close(); Display status and exit; four error boxes	Top-to-bottom sequence; dashed arrows from open(), fstat(), size check and mmap() to error boxes	Normal path and failure exits
5.3	Data Flow Diagram (Level 1)	External entity: User; processes P1 Input and Validation, P2 File Access, P3 Memory Mapping, P4 Read / Modify, P5 Sync / Unmap / Close; data store D1 File on storage	User to P1 to P2 to P3 to P4 to P5; P2, P3 and P5 connected with D1; P4 and P5 back to the user	Data exchanged between the user, processes and the file
5.4	Memory Mapping Workflow	File descriptor and size; choice of mmap arguments; kernel entry; VMA creation; address returned; first access; page fault and page-cache lookup; page mapped in page table	Top-to-bottom sequence	Creation of a mapping and demand paging
5.5	Process Virtual Address Space with Memory-Mapped File	Kernel space; stack; unused; memory-mapped region; unused; heap; data/bss; text; page cache; file on disk	Mapped region to page cache; page cache and file on disk in both directions	Position of the mapping in the address space and its connection to the file
5.6	mmap() Lifecycle	Seven states from file open without mapping to descriptor released	Each state to the next	Time sequence of the resources of the program
5.7	File Access and Synchronization Flow	Read path: load instruction, MMU / page table, page fault, page cache, data available; write path: store, page in memory, page dirty, msync, file storage	Left to right on each row	What happens on a read, a write and msync()
5.8	Module Architecture	Main controller; eight modules (input, opening, information, mapping, access, synchronization, unmapping, closing); error handling and resource management bar	Controller to each module	Division of the program into modules

DOCUMENT PREPARATION CHECKLIST

This page and the next are working aids for the student. They are not part of the report and should be removed before the report is submitted.

Information still to be filled in

- **Roll number** 2520030474,2520030530,2520030451.
- **Academic year** 2026.
- **Guide name** Hema.
- **Hardware details** of the machine used (Table 3.1) and the exact Ubuntu, WSL and GCC versions (Table 3.2).
- **Actual source filename** and executable name: replace `mmap_demo.c` and `[SOURCE_FILE]` / `[EXECUTABLE_NAME]` throughout Chapters 4, 6 and 7.

Screenshots to capture

- The twelve screenshots listed in Table 7.2, in particular the compilation screenshot and the execution screenshot of the actual program.
- A screenshot of `/proc/<pid>/maps` showing the mapped file with `rw-s`.
- Before-and-after `cat` screenshots of the modified file.

Diagrams to create or check

- Eight diagrams are already drawn and inserted (Figures 5.1 to 5.8). Check them for consistency with your actual program; redraw them from Appendix A if you want a different style.
- If your program has a menu, add a menu-structure diagram and update Figures 5.2 and 5.8.

Code snippets to include

- Replace Listing 6.1 and the module excerpts in Sections 6.2 to 6.8 with the real code from your submitted file wherever they differ.
- If your program does not contain the pause for `/proc/<pid>/maps`, the offset and text input, or the preview, update Sections 5.6, 5.7, 6.4, 6.5 and 7.5 to 7.6 so the report does not describe features that are not in your code.

Test results to verify

- Run all sixteen test cases of Table 7.1 on your program and replace each `and` and "Pending" with the real result and Pass or Fail.
- Run the permission test as an ordinary user, not as root.
- Keep the actual program output for Section 7.4 and replace the sample output with it.

Final formatting steps

- The table of contents in this file is a static table with page numbers. After you edit the text, update the page numbers manually, or rebuild the table of contents in Word.

- Check the total page count (target 50 to 70 pages) after inserting your screenshots.
- Add your photograph of signatures or print and sign the declaration and certificate pages by hand.
- Remove this checklist and the viva page before submission.